
University of Portsmouth
BSc (Hons) Computer Science
Third Year

Distributed Systems And Security (DSSEC)

M30225

January 2026 - June 2026

20 Credits

Thomas Boxall
`thomas.boxall11@myport.ac.uk`

Contents

I	Distributed Systems	2
1	Lecture - Module Intro & Overview of DS (2026-02-03)	3
2	Seminar - Introduction to DS (2026-02-04)	8
3	Lecture - Communication Models (2026-02-10)	10
4	Seminar - Communication Models (2026-02-11)	14
5	Lecture - Time & Coordination (2026-02-17)	16
6	Seminar - Time (2026-02-18)	20
7	Lecture - Distributed Shared Memory (2026-02-24)	24
8	Seminar - DSM (2026-02-25)	28
9	Lecture - Web Services and Naming Systems (2026-03-03)	32
10	Seminar - Web Services & Naming Conventions (2026-03-04)	35
11	Lecture - Mobile & Ubiquitous computing (2026-03-10)	39
12	Lecture - Fault Tolerance (2026-03-17)	42
13	Seminar - Fault Tolerance (2026-03-18)	46
II	Security	50
14	Lecture - Foundations of Distributed Systems Security (2026-04-21)	51
15	Lecture - Network Layer Security (2026-04-28)	57
16	Lecture - Transport Layer Security (2026-05-05)	66
17	Lecture - Application Layer Security (2026-05-12)	72

Part I

Distributed Systems

Page 1

Lecture - Module Intro & Overview of DS

📅 2026-02-03

🕒 09:00

👤 Amanda

1.1 Introduction to this Module

This is a module of two parts:

- Distributed Systems taught by Dr. Amanda Peart, first 8 weeks with 1 lecture (1 hour) and 1 seminar (2 hours) per week. Exam to cover this content worth 60% which will be held during March, before the Easter break.
- Security taught by Dr. Fahad Ahmad, which will have a different schedule. The exam to cover this content will be during the summer exam period and worth 40% of the overall module.

For Distributed Systems - there are two books recommended, one by Coulouris which is available in the UoP E-Library and on GitHub; and one by Tanenbaum. Both are relevant in places to supplement lecture material, regardless of their publish date.



There's some more more information on the content to be covered in this Module available on the Moodle page.

1.2 What is a Distributed System?

A *Distributed System* is the interaction between two or more computing based devices which are connected in some fashion. Historically, this would have been seen as two Laptops, or PCs, then more recently as mobile phones, and even more recently as Meta Glasses (etc). This is an ever evolving field, however the two primary issues remain the same:

- How do the devices communicate? For example through which networking protocols and methods
- How do the devices interact? For example through distributed software

Definitions

Distributed System “A distributed system is a collection of independent computers that appear to the users of the system as a single computer” - Tanenbaum.

Distributed System “A system in which hardware or software components located at networked computers communicate and coordinate their actions only by message passing” - Coulouris.

1.3 Why use Distributed Systems?

In short, we want to do things faster. Expectations for technology are increasing - the users demand the service to be instant, the webpage to load immediately, and to never see the loading wheel of

death.

There are three methods which can generally be employed to to do things ‘faster’: working harder; working smarter; and getting help.

1.3.1 Working Harder

Working Harder is the idea of putting more effort into the task, in the hopes of completing it faster or better. For example, the night before exams cramming as much as possible. It can also apply to the thought of finding shortcuts to achieve given tasks. For example, Michaelangelo carrying his lunch up the scaffold with him so he didn’t have to stop painting, or a desperate student taking their lunch to the library to avoid that 2 hour side quest to Greggs.

In computing - we can see working harder through having higher performing hardware.

However working harder alone often isn’t enough.

1.3.2 Working Smarter

Working Smarter is the idea of finding a way to reduce the amount of work needed to accomplish something. Taking a task and making it more efficient or smarter has a positive side effect, in that you get more out of it with less effort. For example, McDonalds have a streamlined kitchen, or a racing line used by Formula One drivers to minimise the distance to cover.

In computing - we can see working smarter through more efficient algorithms.

Although, even if we manage to work harder and more efficiently, that still may not be enough.

1.3.3 Getting Help

Getting Help is the idea of more computers or computational devices, etc being involved in the process to increase capacity. For example, a group of roadworkers digging a hole in the road is more efficient than one person on their own.

In computing - we can see getting help through distributed or parallel processing.

However, there are negative cases of getting help too. For example, some committees spend longer discussing and thinking than doing, where an individual or smaller committee may make the decision in a quicker and less painful method. In saying this, getting help is good, but you must be aware of the risks associated.

1.4 Where are Distributed Systems Used?

Distributed systems can be seen in lots of places. They’re commonly embedded in technology:

- Network File System (NFS)
- SSH / telnet
- Web - retrieving a page or used in CGI Scripts
- Network Printers
- ATM (Cash Machine)

Alternatively in more specialist technology:

- Distributed Shared Memory (DSM)
- Distributed Objects (COBRA or Java RMI)

- Process Migration (Condor)
- Replication (NIS)
- Network Video & Audio

Most of the above systems / applications communicate in some way with other systems, applications or entities. This means they communicate through a network. There are some requirements for the network. (Flashbacks to Networks L4 module).

1.4.1 Networks Requirements

In a traditional layered protocol stack, we may use the OSI or TCP/IP protocol stacks. These have defined layers and defined functions:

- Data link layer - handles the bits on the wire
- Packet switching
- Addressing and routing
- Reliable data streams
- Remote Procedure Calls (RPC) & other high level protocols (SNMP, HTTP, etc)

However in the land of distributed systems, the layered architecture isn't so present; where it's more common for protocols, APIs and services to be mixed and matched as required. Some layers do exist, however:

- RPC, Time & directory & security, File Server
- Storage, Directory, Replicated FS.

1.5 Structure of Distributed Systems

A distributed system can be considered to have three levels: Fundamentals, Distributed OS, and Application.

The *Fundamental Elements* level is the lowest level. This is concerned with the individual processes, threads, and concurrency controls. It also handles the synchronisation between different devices, and the Remote Procedure Calls (RPC). The lowest level also deals with naming the different components for clarity and ease of access, caching and finally network protocols. Many of the topics listed here were covered during the Operating Systems and Internetworking module at Level 5.

The *Distributed Operating Systems* level sits in the middle. This is concerned with scheduling, transactions, replication, fault tolerance, persistence and high availability. Many of these elements are to do with ensuring the multitude of devices in the distributed system are all using and referencing the same data.

The *Application* level is the highest level. It handles any distributed shared memory, distributed objects, manages security and protected environments.

1.6 Issues & Considerations in Distributed Systems

Within a distributed system, there can be a need for *costly communications*. Most commonly this is seen in a setting where Symmetric Multiprocessing (SMP) is not available, leading to multiple discrete but connected nodes needed. The bandwidth and latencies of SMP is unparalleled to that of multiple nodes - causing performance degradation.

Unreliable Communications are a significant issue between distributed systems. If the connection

is unreliable or unavailable altogether, messages may be lost or corrupted which means a remote computer may not be usable.

Distributed Systems need to have resiliency built in; a system should be able to continue its operation after the *independent failure* of a node, preferably without the user noticing any performance issues unless the failure is significant.

The communication between nodes can also be problematic, where *insecure communications* could be exposed leading to unauthorised access, which potentially may be malicious.

1.7 Design Issues

When designing a distributed system, or even a system which may need to be distributed in the future, there are a number of issues which should be considered to ensure the effective functioning of the system.

1.7.1 Naming

Names assigned to resources or objects must have global meanings. This refers to any names or identifiers used in the code or deployments. The names should be independent of the locations of the objects. There should be a name interpretation system that can translate names in order to enable different programs to access named resources. Names used should scale and translate efficiently.

1.7.2 Access and Authorisation

Authentication should be global service across the entire distributed system; users shouldn't need to authenticate for every different component of the same application. The same functions should be usable everywhere and should have reasonable performance.

1.7.3 Communications

The performance and reliability of the communication techniques used for the distributed system are critical to the performance of the system. This means that the data communicated within the system should always be up to date, so definitely not yesterday's data presented as though it's today's.

1.7.4 Software Structure

Open software is achieved through the design and construction of software components with well defined interfaces, such as APIs. Through an open system, more people can improve and access the system, however it does raise security issues through the number of people accessing the system.

Data abstraction within the software is important, as well as designing a framework that can work with existing and new services without duplicating the existing services.

1.7.5 Resource Management

While designing the distributed system, consideration must be given to the deployment. Optimisation of one component of the processing or communication resources within the network can simply move the bottleneck to another component in the system. Load balancing can be employed to move the workload between different systems.

1.7.6 Consistency Maintenance

Within a distributed system there may be multiple nodes performing the same function or needing to access the same data; herein lies issues with consistency. Consistency is concerned with:

- Data - where multiple nodes attempt to access a single data item, will they both get the same item?
- Replication of data - fault tolerance in this
- Cache - performance improvements, FT, availability
- Failure - is there a master / slave architecture setup
- Consistent time
- User Interface - ensuring ease of use and smoothness to users

1.7.7 Security

Authentication ensures users have the right access to resources. Access Control provides services to give different users different levels of access. Auditing keeps logs of all actions taken.

Page 2

Seminar - Introduction to DS

📅 2026-02-04

🕒 11:00

👤 Amanda

The second session each week of this module will take the form of a ‘Seminar.’ Amanda introduced this as a new concept where we are expected to discuss and engage in conversations surrounding the topics rather than cracking on with exercises - as has been seen in most smaller-than-lecture timetabled sessions until now.

Each week, a Question of the Week will be released - this will be a longer-form question to consolidate understanding of the weeks topic. They will be released at some point during the week on Moodle and then discussed at the start of the following week’s seminar.

2.1 Seminar Exercises

Ex. 1: What are the Key Goals of a Distributed System?

- Scalability (size; geography; administration)
- Fault Tolerance (reliability; tolerance)
- Efficiency
- Performance (latency; throughput; resource allocation)
- Data Integrity
- Concurrency
- Transparency (location; replication; failure)
- Replication
- Availability (accessible; replication; fault tolerance)
- Security (authentication; authorisation; encryption; auditing)
- Maintainability

Ex. 2: What are the Challenges with Distributed Systems

- Heterogeneity (Variety and Difference)
 - Networks and protocols
 - Computer hardware
 - Operating systems
 - Programming languages
 - Implementation by different developers (compatibility)
- Openness
- Security (bottlenecks; decentralised systems; metadata growth)
- Scalability (both scaling up and scaling down)
- Fault tolerance (failure handling e.g. node crashes)
- Concurrency and Synchronisation
- Transparency (Access; location; concurrency; replication; failure; scaling)

- Consistency and replication
- Resource Management
- Scheduling (load balancing; synchronisation; clock drift)
- Maintenance (monitoring; debugging)

2.2 Question of the Week

Host Computers used in a peer-to-peer network, in this case traditional desktop computers, are based in users homes or offices.

- 1. What are the implications of this for the availability and security for any shared data objects they hold?**
- 2. To what extent can any weaknesses be overcome through the use of replication?**

Q1. People turn off their computers when not using them; and while even if they are on for most of the time - there will be periods of time when they are off, either when the user is away or in transit. The owner of the participating computer is unlikely to know the other participants so in turn their trustworthiness is unknown. With the current hardware and operating system for this sort of communication - the owner has control over the data on it and may change or delete it. The first issues, around downtime, may be non-issues depending on what the data is that is to be shared over the P2P network; for example music or other media may not be required immediately - so it may be acceptable to wait for the host machine to come back up before the data can be downloaded, although obviously this isn't suitable for all download types.

Q2. If the data replicas are sufficiently widespread and numerous, the probability that all are unavailable simultaneously can be reduced to a negligible level. This still leaves security concerns, however. To ensure integrity of the data objects at multiple hosts (against intentional tampering or accidental error), it's good to perform an algorithm to establish a consensus about the value of the data, such as comparing the object's hash value. Therefore then when the object is received by a client - the hash can be checked for correspondence with the object's ID.

Page 3

Lecture - Communication Models

📅 2026-02-10

🕒 09:00

👤 Amanda

The communication models used in Distributed Systems define how different processes and nodes (systems) interact with each other. The models influence the design of the system, as well as its efficiency and reliability. One model used may feed communications to another model, and therefore a different system.

There are a number of different models which will be explored in this lecture. However this is not an exhaustive list - as that would fill 8 weeks of lectures, not just a 1 hour lecture!

We will never negate all the issues with these communication models, however we can try to improve them. The networks connecting devices together are improving which is inherently improving the models.

3.1 Message Passing

Message Passing works by processes sending and receiving messages over a network. This is commonly used in distributed computing for inter-process communication.

Message Passing can work asynchronously or synchronously to exchange messages. It works both ways so it can best fit the message contents and what needs to happen with the data on receipt. Messages must be serialised before transmission and deserialized on receipt. Messages passing is susceptible to network failures, so some thought has to be given as to how to overcome this when designing the system. Message passing is a form of direct communication.

Typically, message passing is seen where a process needs to communicate with another process. Through the first process passing the second a message - the second process then has a copy of the data. From this arises a need to keep the data consistent across the network.

Examples of applications of message passing:

- Web Sockets (TCP / UDP)
- Remote Procedure Calls (RPCs)
- Message Queue Systems (Rabbit MQ, Kafka)
- REST APIs

3.2 Shared Memory

Shared Memory is a model in which multiple processes / nodes read from and write to a shared memory space. This model is more common in multi-threaded systems but can be used in distributed systems where it becomes the *Distributed Shared Model* (DSM). With the uprising in cloud computing, the memory can now geographically be anywhere.

The Shared Memory model doesn't have a need for message exchange, rather the systems just directly interface with the shared memory. However it does require some form of synchronisation - often seen with Locks or Semaphores (as seen in L5 OSINT).

To the user, shared memory is seen as a single memory location when in reality - this is split across multiple different nodes.

Examples of applications of shared memory:

- Distributed Shared Memory
- Distributed File Systems (Google Drive, Hadoop HDFS)

An issue with shared memory is the need for ensuring that the data is saved on a sensible node. What this means is that if Node 1 is asking Node 2 for some data, where in reality only Node 1 needs this data - then it would be most sensible to store the data on Node 1. This should also be extended to ensure that the data is stored on a node which is geographically close to the users.

3.3 Remote Procedure Calls

Remote Procedure Calls (RPCs) allow a process to invoke a function on a remote system as if it were a local function call. They abstract the details of the communication on the network.

RPCs work through the client sending the request to the server which processes the request and then responds to the client with its response. They can be blocking (synchronous), or non-blocking (asynchronous). The data used in an RPC requires serialisation before it can be transmitted on the network, which is commonly achieved using protocol buffers or JSON.

Within a RPC system - there has to be a clear and consistent naming convention - something which can be understood by both the client and server. On the server side, there may be a stub which can translate received names into local names, alternatively to decide if the data can be request can be processed or not.

Examples of applications of RPCs:

- Google RPC (gRPC)
- Apache Thrift
- Java Remote Method Invocation (Java RMI) - which was one of the big features that made Java explode in popularity

3.4 Publish/Subscribe Model

The Publish/Subscribe (Pub/Sub) model is an event-driven communication method. Publishers broadcast messages without knowing what subscribers (if any) are listening for them.

This is a decoupled architecture, where there is no direct connection between the sender and receiver. Communications are handled asynchronously. Pub/Sub is seen in real-time streaming and event-driven architectures.

Examples of applications of Pub/Sub:

- Apache Kafka
- Google Cloud Pub/Sub
- MQTT

3.5 Data Streaming Model

The data streaming model handles processes continuously sending and receiving data in real time. This is ideal for applications that need low latency data transmissions.

Data streaming often involves continuous data flow working in a push model, where a process continuously pushes data at the receiver who has to deal with the flow in some way. There may be a broker of sorts in the middle of the flow, cleaning the data or merging it with other data streams. This model is suitable for real-time analytics and monitoring systems and can commonly be seen working with Pub/Sub systems.

Due to the relentless data pushing aspect of the model - there is a need for a lot of bandwidth to handle the traffic and some decent infrastructure for the system to run on. We strive for there to be zero latency in this system in case we are looking to make critical decisions on realtime data. There may also be some storage for archived data, which is separate to working storage.

Examples of applications of Data Streaming:

- Apache Kafka
- Apache Flink
- Spark Streaming

3.6 Peer-to-Peer

Peer-to-Peer (P2P) is a model in which all nodes in the system act as both a client and server. They share resources and data without a central authority commanding and conducting the way they work.

P2P is a decentralised architecture, it is easier to bring in nodes and ensure replication between them. It is highly scalable but may have inconsistency issues between nodes. P2P is used in blockchain, file sharing and decentralised applications.

In a way, utilising P2P is almost like utilising shared memory. However there is a risk that if one node goes down and there is no suitable data replication in place, the entire system could lose access to the data that node is holding.

In the early days of P2P systems, back to the Limewire era, there were a number of issues with the P2P software utilising too much of the PCs resources, making the system unusable for the user.

Examples of applications of P2P:

- Bit Torrent
- Blockchain
- Skype

3.7 Tuple Space Model

The Tuple Space Model is an example of coordination based communication, where the processes interact through a shared tuple space. The processes read, write and take data tuples instead of direct message exchange.

This model is based on the Linda Coordination Models. It is decoupled in time and space as the processes don't need to be online simultaneously - however does require some coordination. It is used in distributed coordination and parallel computing. This can be more efficient, however there are complexities with managing the tuples effectively.

Examples of applications of Tuple Space:

- JavaScript
- Apache ZooKeeper

- Tibco Rendezvous

3.8 Cons With These Systems

There are of course cons with using these communication models:

- Message Passing - requires message handling
- Shared Memory - synchronisation overhead
- RPC - can be blocking to other processes / applications
- Pub/Sub - latency in event delivery
- Data Stream - complex implementation
- P2P - data consistency issues
- Tuple Space - complexity in managing tuples

Page 4

Seminar - Communication Models

📅 2026-02-11

🕒 11:00

👤 Amanda

A Synchronous Distributed System has known lower and upper bounds on the: time to execute each step of a process; time to receive a message transmitted over a channel; and time by which a local clock will drift from the real time within a process. The sender will wait for the receiver to process and respond before continuing. This is simpler to coordinate however has increased latency.

An asynchronous DS has no bounds on process execution speeds, message transmission delays or clock drift rates. The sender will proceed without waiting for an immediate response. This is more efficient, however it's harder to handle failures.

4.1 Seminar Exercises

Ex. 1: Why is communication critical in Distributed Systems?

- Data Sharing between processes
- Synchronisation and Coordination
- Fault Tolerance and Consistency

Ex. 2: What are the advantages of Remote Procedure Call (RPC)?

- Makes a remote call look local
- Simplifies Distributed Programming

Ex. 3: What are challenges faced by Distributed Systems Communication Models?

- Latency (overcome by efficient protocols)
- Fault Tolerance (overcome by replication and retries)
- Security (overcome by encryption and authentication)

Ex. 4: What communication model(s) does Google Drive use?

- Client-Server Model: Primary model for file access and synchronisation
- Message Passing Model: Used for file synchronisation across devices
- Publish-Subscribe Model: Event-based updates for real-time collaboration
- Peer-to-Peer (P2P) Model: Direct Communication in Google Docs collaboration
- Remote Procedure Call Model: Used for efficient communication between services

4.2 Question of the Week

A university is developing a cloud-based learning management system (LMS) that allows students and staff to perform a number of tasks.

For the following requirements, identify which communication model(s) would be most

suitable for each feature and justify your choices:

1. Uploading and downloading lecture materials (e.g. PDFs, slides)
2. Receiving real-time notifications for new assignments
3. Collaborating on shared documents in real time (for group projects, etc)
4. Streaming live lectures with interactive participation ¹

Q1. Uploading and downloading lecture materials would use the client-server communication model. This is because the LMS needs a centralised cloud storage which both students and staff can access. The client-server model would be utilised as the staff & students acting as clients while the LMS itself is the server.

Q2. Realtime notifications would use the Pub/Sub model. This works because students would be able to receive an instant update when a member of staff posts a new assignment. The Pub/Sub model is implemented with the LMS publishing to the broker when a new assignment is published which then the subscribers (students) are subscribed to so they can see the notification.

Q3. Collaborating on shared documents in real-time would use the peer-to-peer model. This method works because it is more efficient than constantly requesting updates from a central server enabling a more real-time shared document experience.

Q4. Streaming live lectures with interactive participation would use the message passing model. This works because the live streaming element requires a continuous data exchange between the presenter (lecturer) and the student. The message passing model is suited for this because it uses data packets. There may be bi-directional information exchange for example lecturer streaming video content to students while students respond with chat messages and/or reactions. The live lectures might also use Pub/Sub to notify when a live lecture is starting or alternatively use the client-server model for the live video streaming component.

¹QotW paraphrased to reduce repetition

Page 5

Lecture - Time & Coordination

📅 2026-02-17

🕒 09:00

👤 Amanda

Within Distributed Systems, time is crucial. Time underpins operations, ensuring different distributed components are synchronised with each other. However *time* is a reasonably abstract word; so there's a need to define this concept to ensure coordination and interoperability between different systems.

It's entirely expected for two different components in a DS to have some delay between them. Nothing's perfect, and networks certainly aren't. However this delay is important; while some components have a high tolerance for latency (such as file transfer or email transfer), some components have a low tolerance for latency (such as real-time data transfer or financial transactions). From this delay comes the need to synchronise, so if two systems go 'out of sync' - they can be brought back into sync for subsequent operations.

5.1 Synchronisation

Synchronisation is needed to order events and to achieve a consistent global state of the system.

In a simple real-world example, clock synchronisation is especially important in situations such as online bookings, online shopping, stock market transactions, and multiplayer games. In essence - this is anything where there is a finite resource being accessed by multiple users or where the order of operations is vital such as the order in which stocks were bought and sold, or the order in which dragons were slayed.

Synchronisation is a necessary part of cooperation by way of sharing resources, ordering of events and distributed agreements. The possible effects of not synchronising clocks can include issues with elections, mutual exclusions being triggered and distributed transactions failing or becoming out of order.

5.2 Clocks Are Inherently Bad At Keeping Time

Clocks are not accurate. They like to go out of sync either through speeding up compared to the perfect clock, or slowing down as compared to the perfect clock. Even crystal-based clocks used in computers are subject to *clock drift* which means they count at different rates therefore will eventually diverge.

The instantaneous difference between the readings of any two clocks is called their *skew*.

Listed below are some examples of how much clocks drift:

- Standard quartz clocks drift by about 1 second in 11-12 days (approx 10^{-6} seconds/second)
- High precision quartz clocks drift by about 10^{-7} or 10^{-8} seconds/second

The quartz crystals are susceptible to physical variations in the crystals, variations in temperature, etc. The best atomic clocks have a drift rate of one second in 10^{13} seconds (about 300,000 years).

5.3 Synchronisation Methods

There are a number of different methods which can be used to synchronise clocks.

5.3.1 Synchronising Physical Clocks

A computer's clock C_i is synchronised with an external authoritative time source S :

$$(S(t) - C_i(t)) < D \text{ for } i = 1, 2, \dots, N \text{ and for all real times over an interval } I$$

Alternatively, a pair of clocks can synchronise with each other using *internal synchronisation*:

$$(C_i(t) - C_j(t)) < D \text{ for } i = 1, 2, \dots, N \text{ over an interval } I$$

Clocks which are internally synchronised are not necessarily externally synchronised.

5.3.2 Synchronous Systems

A *synchronous* distributed system has known bounds for the clock drift rate; the maximum message transmission delay; and the time to execute each step of a system

Synchronisation works as follows:

1. One process p_1 sends its local process time t to process p_2 in a message m
2. p_2 could set it's clock to $t + T_{trans}$ where T_{trans} is the time to transmit m
3. T_{trans} is unknown, but the uncertainty of it can be calculated using $u = (max - min)$ where min is estimated and max is known.
4. So if we set the clock to $t + \frac{(max-min)}{2}$ then the skew is at most $\frac{u}{2}$.

5.3.3 Cristian's Method

Cristian's method is a method for synchronising time in an asynchronous system. The algorithm is probabilistic, and only really works well if the round trip time is short.

Cristian's method works by, a time server S receiving a signal from a UTC source. This uses external synchronisation when the time-server process S supplies the time according to it's clock. The round-trip time is used to estimate the delay of the packet and therefore the time. However, Cristian's method is susceptible to a time server failing and it does not deal with faulty servers well.

The point at which the client sends the request for time to the server is known as t_0 . The point at which the client receives the response is known as t_1 . The client also knows the minimum transit time (min).

From this information, the client can estimate the time using $\frac{t_1 - t_0}{2}$

Which comes with an estimated accuracy of $\Delta = \pm \frac{t_1 - t_0}{2} - min$

5.3.4 Berkeley Algorithm

The Berkeley algorithm works with a master polling to collect clock values from other devices (slaves). The master uses the round trip time to estimate the slave's clock values. The master then computes the average time in the network and sends the required adjustment to the slaves. Forward adjustments can be done with a jump, however backwards adjustments require a gradual slow down.

5.3.5 Network Time Protocol

The Network Time Protocol (NTP) uses reference clocks as a baseline. These are in UTC and are often atomic clocks run by government agencies such as the Shortwave Radio Station (WWV) run by the National Institute of Standard Time (NIST). These radio stations broadcast short reference pulses at the start of each UTC second (+/- 10msec resolution).

The hierarchical server structure has a tree-like structure where a primary server exists at the root, with end-devices at the leaves. Each layer in-between is known as a stratum.

There are three different modes of synchronisation within NTP, all relying on UDP:

- Multicast: acceptable for high-speed LAN
- Procedure-call: working like Cristian's Algorithm
- Symmetric: between a pair of servers

NTP's connections are re-configurable as required, which increases fault-tolerance. It's scalable for both clients and servers; allowing clients to re-sync frequently to offset drift. NTP also supports authentication of trusted servers, and validation of return addresses to ensure time is only being given to authorised users and authorised users trust where they're getting the time from.

NTP provides a synchronisation accuracy of tens of milliseconds over Internet paths, and about 1millisecond on LANs.

5.4 Logical Time & Logical Clocks

As we already know - clocks cannot be synchronised perfectly within a distributed system. So an idea: what about if we abandon any idea of physical time? Lamport (1978) explored this concept and from here - logical (virtual) time was born. Logical time provides consistent even ordering. Within logical time, *casual ordering* is used - which makes use of the linear idea where an event happens after another event.

To fully realise this - we must first look at a process interaction model:

- A distributed system is defined as a collection of N processes (P) where $P_i, i = 0, 1, 2, \dots, N - 1$.
- Each process $P - i$ has a state s_i which includes values of all it's variables
- The processes communicate only by messages (via a network)
- Each process can send, receive messages and change it's own state

The *event* is the occurrence of a single action.

Events at a single process P_i can be placed in a total ordering; which is denoted by the relation $\rightarrow_i$ (happened before) between the events. For example $e \rightarrow_i e'$ if and only if e occurs before e' at P_i . A history of process P_i is a series of events ordered by $\rightarrow_i$ for example:

$$(P_i) = h_i = \langle e_i^0, e_i^1, e_i^2, \dots \rangle$$



There's a graphical example of this in the slides available on Moodle.

5.4.1 Lamport's Logical Clocks

Okay, so back to Lamport. Lamport explored that a *logical clock* is a monotonically increasing software counter; where each process P_i has a logical clock L_i which can be used to apply logical timestamps to events.

LC1 is where L_i is incremented by 1 before each event at process P_i .

LC2 has two options:

- When process P_i sends message m it piggybacks, therefore $t = L_i$
- When process P_j receives (m, t) it sets $L_j = \max(L_j, t)$ and applies LC1 before time-stamping the event receive m .

A shortcoming with Lamport's clocks is where two events happen concurrently - the Logical clocks may not be equal despite two events taking place at the same time, due to each process having its own logical clock. For example $L_e < L_{e'}$ does not imply $e \rightarrow e'$ which means we cannot spot concurrency.

5.4.2 Vector Clocks

Vector Clocks were designed to overcome the shortcomings of Lamport's clock. The Vector clock V_i at process P_i is an array of n integers.

The big shortcoming Vector clocks overcome is that in a Lamport clock $L(e) < L(e')$ does not necessarily mean that $e \rightarrow e'$. This can be shown in Vector clocks clearer through the vector representation of each process / event. In the same vain, Lamport clocks cannot spot concurrency; however Vector clocks can do.

Vector clocks use vectors to store information about preceding operations and current operations timings in the logical vector clocks.



Further information on Vector Clocks is available in the slides on Moodle

Page 6

Seminar - Time

📅 2026-02-18

🕒 11:00

👤 Amanda

Mutual Exclusion ensures that only one process can access a shared resource (e.g. a file, database, or printer) at a time which prevents conflicts. In a centralised system, this is simple to manage - a central coordinator server manages access to the resource as follows:

1. A process sends a request to the coordinator
2. If the resource is free, the coordinator grants access; otherwise it queues the request
3. Once the process finishes, it notifies the coordinator, when when grants access to the next process in the queue

This simple method is fair and avoids conflicts. It's reasonably easy to implement and maintain which is a key strength. However, due to the nature of a single coordinator - this is single point of failure; which means that if the coordinator fails, then the entire system stops working.

Example: Centralised Coordination

In this example we see three processes: P_A , P_B , and P_C and one coordinator: C .

At first the three processes request in turn to the coordinator to access the shared resource:

$$\begin{aligned} P_A &\xrightarrow{\text{request}} C \\ P_B &\xrightarrow{\text{request}} C \\ P_C &\xrightarrow{\text{request}} C \end{aligned}$$

As P_A reached the coordinator first, it is granted access to the resource first. Processes P_B and P_C are enqueued onto the waiting queue Q .

$$\begin{aligned} C &\xrightarrow{\text{grant}} P_A \\ Q &= P_B, P_C \end{aligned}$$

Once P_A has finished with the resource, it notifies the coordinator to release the mutual exclusion. The coordinator then identifies the process at the front of the queue and grants that access to the resource and removes them from the queue.

$$\begin{aligned} P_A &\xrightarrow{\text{release}} C \\ C &\xrightarrow{\text{grant}} P_B \\ Q &= P_C \end{aligned}$$

This cycle repeats unendingly.

In a decentralised system, there is no single coordinator to control access to shared resources. In this paradigm - multiple nodes work together using distributed algorithms to enforce mutual exclusions. There are two different approaches to how this could work:

- Voting Based Approach - Processes send requests to multiple nodes and they must receive a majority vote (which means they are granted permission) before accessing the resource
- Token Based Approach - A unique token is passed between processes, and only the process holding the token can access the resource.

This is better than a centralised system's approach as there is no single point of failure which makes it more reliable; however the decentralised systems approach is more complex and additionally requires synchronisation between nodes which adds overhead to the performance.

Example: Voting-Based Mutual Exclusion

In this example we have four processes: P_A , P_B , P_C , and P_D .

When a process, P_A in this example wants to access the resource - it sends a request to all other nodes:

$$\begin{aligned} P_A &\xrightarrow{\text{request}} P_B \\ P_A &\xrightarrow{\text{request}} P_C \\ P_A &\xrightarrow{\text{request}} P_D \end{aligned}$$

If the majority agree, in this case that's P_B and P_C , then P_A gets access to the resource.

$$\begin{aligned} P_B &\xrightarrow{\text{grant}} P_A \\ P_C &\xrightarrow{\text{grant}} P_A \end{aligned}$$

However if the majority does not agree - in the event that there are multiple nodes all wanting to access the resource and each voting node votes for a different node; no nodes get access and a second round of voting is used. This is a uncommon but likely scenario to be in.

The voting based approach is slow due to the multiple approvals required for a node to be granted access to the resource.

Example: Token-Based Mutual Exclusion

In this example we see three processes P_A , P_B , and P_C .

This example starts with P_A having the token which it then passes to P_B .

$$P_A \xrightarrow{\text{pass token}} P_B$$

This process uses the resource which is token locked then once it's completed its computation - it passes the token to P_C .

$$P_B \xrightarrow{\text{pass token}} P_C$$

Token-Based MutEx is faster than Voting-Based MutEx as processes only need to pass the token between them. However if the token is lost - mechanisms are needed to validate this loss and then recover the token.

6.1 Question of the Week

NovaTrade is a global, real time electronic trading platform with servers located in London, New York, Tokyo, and Sydney. Each data centre runs multiple distributed services that must agree on the ordering of events, maintain consistent timestamps for financial transactions, and ensure mutual exclusion when updating shared ledgers.

The following constraints apply:

- Each server maintains its own hardware clock with clock drift and clock skew (as discussed in Lecture 3).
- Network latency between regions varies and cannot be perfectly predicted.
- NovaTrade must maintain:
 - Accurate physical timestamps for auditing and regulatory requirements.
 - Consistent ordering of events even when physical clocks disagree.
 - Safe mutual exclusion for updates to shared global assets (e.g., account balances, order books).

Recently, several issues have emerged:

- Trades executed in Tokyo appear (based on timestamps) to occur before earlier trades from London.
- A leader node responsible for time synchronization in the London region crashed, disrupting timestamp accuracy.
- Occasional conflicts arise when simultaneous write requests are sent from multiple regions to the global ledger.

Questions

1. Recommend which time algorithm (Cristian's, Berkley, NTP) the system would benefit more from and justify your answer.
2. Because physical synchronization is imperfect, NovaTrade wants to ensure that all trades are consistently ordered, even when clocks drift apart. Discuss whether NovaTrade should use Lamport or Vector clocks for transaction ordering, considering overhead and accuracy.
3. Recommend the most appropriate method for NovaTrade's global ledger, considering network latency, fault tolerance requirements, and fairness

Q1. NPT. This is too high-risk of a system for the time to go out of sync within in regards to the damage that inaccurate timestamps would cause. Due to the large geographical areas between the different locations - there will be a large amount of latency which means Cristian's algorithm isn't suitable. As there are concerns that a node has the wrong time and is using this as the truth - this would make Berkley's algorithm useless too as the one time which is incorrect would skew the algorithm and cause the other clocks to go out of sync.

Q2. Vector clocks would be better than Lamport clocks as they can accurately convey the order of two concurrent processes where Lamport clocks cant. This is better for financial transactions as the order of operations is vital. There is, however, a larger overhead to be contended with as each vector have to be transmitted and may be substantial in size if there are multiple process. This tradeoff is worth it for the increased accuracy.

Q3. The global ledger would need to be a central object which all nodes can interact with. This

would best fit the Message Passing paradigm where the clients send the ledger server update or read requests which the server processes. The time synchronisation between the clients and server would be of vital importance here to ensure that the ledger is in the correct order, especially given this is handling financial transactions.

Page 7

Lecture - Distributed Shared Memory

📅 2026-02-24

🕒 09:00

👤 Amanda

There will be an additional lecture and seminar combination added on the Tuesday of Consolidation week to make up for Amanda being unable to teach in the first week of the semester. This can't be in the proper teaching time because timetabling restrictions.

7.1 Introduction

Distributed Shared Memory (DSM) is a paradigm in which the memory (both the RAM-esque and longer term storage) used by the Distributed System (DS) is distributed between two or more nodes. This is particularly prevalent in cloud computing, or multiprocessor systems.

This abstraction used for data sharing between different nodes who do not share physical memory allows multiple computers in a distributed network to share memory as if it were a single, unified address space; as well as enabling processes on different machines to read and write to a shared space transparently as if they were running on a single machine.

This works through instead of passing messages between processes, DSM allows nodes to read and write to a shared memory as though they are accessing local RAM. This simplifies programming, eliminating the need for developers to consider message passing - rather they just have access to the shared space. It also improves resource sharing which can be especially useful when a large data set doesn't have to be duplicated on every node. DSM also reduces communication overhead, mostly achieved through reducing / eliminating message-passing request from the network.

There are a number of key features of DSM:

- Transparency - the users don't need to know that the memory is distributed; it also comes up a lot in distributed systems in general
- Scalability - where more nodes can be added or removed without affecting performance
- Consistency Control - ensures all nodes see updated memory values correctly; however this presents some other challenges as we'll see later in this lecture
- Synchronisation - uses locks, semaphores, or barriers to prevent conflicts

We can see DSM in the real world in situations such as Google Drive. This has a number features such as collaborative edits where consistency is crucial, in that all concurrent editors of the same file need to always have the latest version. Alternatively multiplayer games require synchronisation of a shared world state between different users.

From the above examples - we can see how important consistency is. Consistency, especially how good we make it is one of the grey areas in DSM as there is a tradeoff between performance and consistency. The higher levels of consistency the DSM maintains, the more significant performance implications this has; and vice versa.

7.2 DSM Models & Architecture

DSM models come in two shapes: *centralised* and *decentralised*. The centralised architecture refers to where a single server manages the shared memory. Access to the data on that server is only via that central server as the data is all in one place. This is simple to manage, and offers easier consistency control; however does introduce a significant single point of failure. Meanwhile, decentralised architecture refers to where memory is spread across multiple nodes (which appear as one after abstraction), with each node managing part of the memory. This is better than centralised as there is no bottleneck and higher fault tolerance, however it is harder to manage consistency.

The decentralised architecture can be seen in a number of models:

- Page based
- Object based
- Variable based
- Library based

Paged based DSM is the simplest of the models. It works much like virtual RAM in an Operating System; in that pages are mapped across different nodes in the system. Each page is present exactly once on each machine. A reference to a local page is done in hardware which grants it full memory speed, however this is using fixed size pages. Any attempt to reference a non-local page causes a hardware page fault which traps to the OS, which then sends a message to the remote machine which finds the requested page and sends it to the requesting processor. After which the faulting instruction is restarted and it can complete. This is efficient for large data sets, however can exhibit poor performance. There are high overheads especially as pages are sent back and forth and it poses increased complexity for memory synchronisation. The challenge with this model is to minimise the network traffic it generates and to reduce the latency between the moment a memory request is made and the moment it is satisfied.

Object-based DSM works through dividing the memory into user-defined objects instead of pages. These are a collection of language level objects which require serialisation & synchronisation. The objects are then stored on different nodes, and processes request them as needed. The system ensures that only one copy of an object is updated at a time. Interaction with the objects are limited to the object interface, providing more access control. This is more efficient than page based DSM as it reduces unnecessary data transfers and has more embedded access control. However managing the consistency can be complex. It requires advanced synchronisation mechanisms to reduce errors and using methods such as Java RMI can reduce performance.

Shared Variable DSM works through sharing only certain variables and data structures that are needed by more than one process. This changes the challenge from how to page over the network, to how to maintain a potentially replicated distributed database consisting of shared variables. This model processes shared specific variables rather than entire memory pages or objects. The variables which are shared are synchronised automatically across the different nodes. There are three types of variable: ordinary, shared and synchronisation. Shared variables are sequentially consistent if used within critical regions. This model is simple to program and efficient for small amounts of shared data, however it is not scalable and the synchronisation can become complex.

Library Based DSM utilises communication between the instances of the language at run-time. The processors make library calls inserted by a compiler when they access data items in DSM. The libraries access local data items and communicate as necessary to maintain consistency.

7.3 Consistency Models in DSM

There are a number of different consistency models, which define the level of consistency required and how that consistency is achieved, within DSM:

- Strict Consistency
- Sequential Consistency
- Causal Consistency
- Eventual Consistency
- Weak Consistency

All of the models have trade-offs between performance and consistency. With all the models, the goal is to ensure that all nodes have a consistent view of shared data while balancing performance and synchronisation overhead.

Strict Consistency is where all processes see memory updates immediately. This is the strongest level of consistency and ensures correctness in critical applications. However there are high overheads with this given the constant level synchronisation requirements.

Sequential Consistency is where all processes see operations in the same order as relative to logical time. This is easier to implement as it is not reliant on physical time and it ensures the correct order of operations. However, it is reliant on synchronisation which may take some (admittedly small) amount of time to take place - so is not ideal for performance sensitive applications.

Causal Consistency is where related (causally dependent) writes must be seen in order and immediately, but independent writes can be seen in any order. Within a distributed system, the variables (or data points) will be divided into independent and dependent. The dependent variables have to be updated immediately and they will often feed into independent variables. This means that the dependent has to be updated before the independent can be. It may occur that at some points in time - the independent variables stored on each node will be inconsistent but they will eventually converge.

Eventual Consistency is where if no new updates occur - all processes will eventually see the same data (this could be a few milliseconds, minutes or longer). Temporary inconsistencies are allowed (tolerated) but over time all nodes converge to the correct state. This is best for highly scalable distributed systems as it improves performance by reducing synchronisation overheads. However the temporary inconsistencies may make this unsuitable for some DSM applications.

Weak Consistency, as the name suggests, is where the system does not guarantee immediate consistency after each write. Instead, consistency is only enforced at synchronisation points (i.e. when a process explicitly requests the latest data). This reduces synchronisation therefore improves the performance and works well when immediate accuracy is not required; however it does introduce the need to handle out-of-date data. It also presents some challenges with predicting the data correctness within the DS.

With the need for consistency between nodes in DSM, comes the need for synchronisation. As part of the synchronisation process - there needs to be some way for one node to signal to other nodes that it now has either up-to-date data or out of date data so that nodes don't use out of date data.

7.4 Implementations of DSM

The *hardware-based* implementation of DSM uses specialised hardware to handle load and store instructions applied to addresses in DSM, and to communicate with remote memory modules as necessary in order to implement them.

The *page-based* implementation of DSM implements a region of virtual memory occupying the same address range in the address space of every participating process.

The *library-based* implementation of DSM is different. In this type of implementation, sharing is not implemented through the virtual memory system but by communication between instances of the language run-time. Processes make library calls inserted by a compiler when they access data items in a DSM. The libraries access local data items and communicate as necessary to maintain consistency.

It is also be explored to share only selected portions of the entire address spaces, more specifically - just those variables or data structures that need to be used by more than one process. Alternatively to replicate the shared variables on multiple machines, sharing variables instead of pages, which means potentially reads can be done locally without network traffic and writes can be done using multi-copy update protocols. It's also being explored to share encapsulated data types (objects) including methods using Remote Method Invocation.

7.4.1 Ring-based Multi-Processor With Shared Memory

The *Ring-Based Multi-Processor With Shared Memory* model has no centralised global memory, rather its divided into private and shared data. Memory is shared between nodes in a token-ring fashion in 32-byte blocks. Each block has a fixed home.

To read a word from the shared memory, the memory address is passed to the device which checks the block table to see if the block is present; if it is - then the request is satisfied. If not present, then the device waits until it captures the circulating token then puts a request packet onto the ring. As the packet passes around the ring, each device checks if it has the requested block. If it does have the requested block, it provides it and clears the exclusive bit if set, and then when the token returns to the requester - it always has the requested block attached.

Writing is similar - in that if the block to be written to is present and is the only copy (exclusivity bit is set) the word is written locally; otherwise if it not the only copy, an invalidation packet is sent around the ring to invalidate all other copies then the exclusivity bit is set and the write proceeds. However, if the writing process doesn't have the block at all, they send out a read request and an invalidation request - so by the time the token gets back round to them, they have the only copy of that block so they can set the exclusivity bit and write their word.

7.4.2 NUMA Multi-Processors

Caches are a common example of a DSM wherein if the data changes in the master location, all caches have to update to serve the master. This is simply done through a write-invalidate which invalidates the caches, then subsequently a write-update is issued which updates all the caches to be in line with the master.

However, hardware caching in large multi-computer DSs is not simple. This means that large multi-computers are expensive and not in widespread use. An alternative to this is *non-uniform memory access* (NUMA). NUMA machines access remote memory much slower than local memory. This means that a CPU can directly execute a program that resides in remote memory but the program may run slower (than it would in local memory).

NUMA provides a single virtual address space which is visible to all CPUs. There is no attempt to hide poor performance of non-local memory via caching, however it can replicate read-only pages or migrate read-write pages to local memory. A page-scanner daemon is used to watch references, and it freezes a page in place if it migrates too much. The page is invalidated if it is being accessed much more often by another cluster.

Page 8

Seminar - DSM

📅 2026-02-25

🕒 11:00

👤 Amanda

8.1 Seminar Exercises

Ex. 1: Match the DSM models with their descriptions

- Page Based - A reference to a local page is done in hardware at full memory speed
- Object Based - Interaction is limited to the interface which provides more access control
- Library Based - Communication is done at runtime
- Variable Based - Comes in three types, ordinary, shared and synchronisation

Ex. 2: Match the DSM models with its application

- Page Based - NUMA
- Object Based - Java RMI, Cobra
- Library Based - MPI
- Variable based - Open MP

Ex. 3: Which consistency model is best for a global cloud storage system?

Eventual Consistency is used because it is highly scalable, has high availability, it optimises performance of consistency and reduces overheads. Critically, eventual consistency is the best balance of performance, scalability and consistency for the application.

8.2 Question of the Week

You have been hired as a systems engineer for DataSphere, a company building a globally distributed collaboration platform called CollabCloud. The platform supports real-time document editing, shared dashboards, and distributed analytics services used by teams located across Europe, the US, and Asia.

To support these features, CollabCloud uses a Distributed Shared Memory (DSM) abstraction. Different components of the system use different DSM models depending on performance and consistency needs:

- A page-based DSM supports large, shared research datasets accessed by geographically distributed analytics clusters.
- A library-based DSM handles replicated Python data structures used by developers during real-time collaborative coding.
- An object-based DSM supports shared document objects (paragraphs, tables, images) edited by users in real time.
- A variable-based DSM handles shared counters, flags, and small metadata values used for coordination.

Because of user expectations for responsiveness and correctness, different consistency models are applied per subsystem:

- Strict consistency is considered for audit-critical financial data.
- Sequential consistency is used for shared dashboard updates.
- Causal consistency is being evaluated for collaborative editing sessions.
- Eventual consistency is used for user presence and “who is online” indicators.
- Weak consistency is used for analytics workloads where full accuracy is not always required.

Questions:

1. Explain how strict, sequential, causal, eventual, and weak consistency differ in terms of: visibility of writes; ordering guarantees; typical network assumptions; performance implications in a wide-area distributed system such as CollabCloud. Use examples from the scenario to illustrate your points.
2. For each DSM model used in CollabCloud (page-based, library-based, object-based, variable-based), identify one appropriate consistency model and justify your choice. Your justification should consider: granularity of sharing; access patterns; communication overhead; potential for false sharing; user expectations for accuracy vs. performance.
3. During testing, analytics clusters running on the page-based DSM experience severe performance issues due to excessive page thrashing¹ whenever sequential consistency is enforced. Explain:
 - Why page-based DSMs are particularly sensitive to consistency traffic.
 - Why sequential consistency may be contributing to the observed performance degradation.
 - Which weaker consistency model would improve performance while keeping data “usefully correct” and why.
4. The collaborative document editing feature uses object-based DSM. Management has asked whether causal consistency is strong enough for real-time editing. Discuss whether causal consistency is suitable, by considering: the causal ordering of edits; the potential for conflicting concurrent operations; perceptions of consistency; whether a stronger or different model (e.g., CRDT-based eventual consistency) might be more appropriate.
5. The variable-based DSM is used for global counters (e.g., “total edits today”), feature flags, and distributed configuration values. Evaluate the risks of using eventual consistency or weak consistency for these metadata variables. Provide concrete examples from the scenario where these risks could cause operational problems.

Q1. Strict Consistency:

- Visibility / ordering - every read returns the value of the most recent write according to a real-time global clock order
- Network assumptions - requires globally synchronous clocks and instantaneous propagation which is impractical in real DS
- Performance - extremely expensive due to coordination, unsuitable for wide area network like CollabCloud

- Example - financial audit data might need this in theory but the global distribution makes it unrealistic

Sequential Consistency:

- Visibility / ordering - all processes observe writes in the same order but not necessarily real-time order
- Network assumptions - does not require global clocks but requires global agreement on write order
- Performance - high communication costs, global ordering reduces concurrency
- Example - dashboard updates must appear in a consistent order to all users

Causal Consistency:

- Visibility / ordering - operations that are causally related must be seen in the same order everywhere, but concurrent operations may be observed in different orders
- Network assumptions - tracks causal dependencies, allows more concurrency
- Performance - more efficient than sequential as it avoids unnecessary synchronisation
- Example - document edits; if user A writes a sentence and user B edits it, all users must see A's edit before B's edit

Eventual Consistency:

- Visibility / ordering - no guarantees about ordering, replicas will converge eventually once updates stop
- Network assumptions - designed for highly distributed, weakly connected systems
- Performance - very high availability and low latency
- Example - 'who is online' indicators which can tolerate temporary inconsistencies

Weak Consistency:

- Visibility / ordering - no ordering guarantees at all, updates will propagate whenever the system decides
- Performance - fastest but least intuitive
- Example - background analytics where exact intermediate results are not critical

Q2. Page-Based DSM would best suit Eventual or Weak consistency models because large pages cause false sharing and the requirements for higher consistency models would trigger frequent invalidations. This works because analytic workloads often tolerate slightly stale data and the performance requirements outweigh the strict ordering.

Library-Based DSM would best suit Sequential Consistency because where the developers use shared data structures requiring predictable semantics, the library DSM gives fine control over read/write functions.

Object-Base DSM would best suit Causal Consistency because edits to shared document objects often have meaningful causal relationships therefore there is a need for the edits to propagate correctly without imposing a global total order.

Variable-Based DSM would best suit Eventual Consistency for non-critical metadata because small variables like presence flags or counters have relaxed correctness requirements and they are naturally convergent and low-risk.

Q3. Page-based DSMs are sensitive to consistency traffic because large blocks of memory are moved at a time, even if only one byte changes - the entire page has to be invalidated and retransferred. Distributed analytics jobs often perform fine-grained updates on shared pages which cause heavy “page thrashing.” Performance issues are made worse by sequential consistency because every write must be seen globally in the same order, which is implemented as every replica must invalidate or update all copies of the page. Given the large geographical span between Europe - US - Asia, this problem is amplified. A better choice would be eventual or weak consistency, especially because analytics tasks can tolerate slightly stale data, therefore reducing page transfers and improving throughput while preserving ‘useful correctness.’

Q4. Causal Consistency is good because it guarantees that causally dependent edits are applied in the correct order and avoids heavy global synchronisation. However concurrent updates to the same object may conflict and causal consistency alone does not give deterministic conflict resolution; users may therefore see temporary divergence. A more appropriate alternative would be to use eventual consistency where real-time editors use conflict-free replication data types, allowing concurrent edits to automatically converge without conflict; this provides a strong user-perceived consistency without enforcing global ordering.

Q5. Eventual / weak consistency may lead to inaccuracies on analytic dashboards or billing systems due to data divergence. Similarly, flags enabling new features may not propagate before the corresponding configuration value has arrived, which may lead to some nodes enabling a feature without the required dependencies which would lead to a system crash. Updating configuration variables using eventual consistency can cause nodes to run incompatible settings, which could lead to instability or inconsistent document processing behaviour. Critical Metadata should use sequential consistency, or at least causal consistency.

Page 9

Lecture - Web Services and Naming Systems

📅 2026-03-03

🕒 09:00

👤 Amanda

The make-up lecture taking place next week is at 09:00 in PK 1.23 (same as the standard series). There will only be two seminars next week, both on Tuesday.

9.1 What Is A Web Service?

A web service is a collection of independent software components, or nodes, located on different machines that interact via standard web protocols to function as a single, unified system.

Web services have an abstraction layer which sits between the two devices who are attempting to communicate together. This application layer serves as middleware converting between two (potentially different) platform-specific technology stacks. For example the client might use JavaScript to request some data from a web service who uses PHP to construct the response - the communication between these two services is platform and language agnostic to ensure interoperability between different devices.

There are three components to a web service:

- Service Providers create and offer the web service. They describe their web services in a standard format which is published in a service registry.
- Service Registries contain data of the service provider. They contain the address and contact details of the provider as well as technical details for what the provider is and how it should be interfaced with.
- Service Consumers firstly retrieve information from the registry. They then use this information to bind and invoke the web service on the service provider.

Generally within web services, we'll be talking about decentralised systems - so we should be assuming there are multiple service providers and multiple service registries.

Web Services allow for programs to be written in different languages on different platforms to distribute in a standard base manner. They adapt the loosely coupled web programming module for use in applications that are not application based. The goal is to provide a platform for building distributed systems using software which: runs on different operating systems and devices; has been written using different programming languages and tools from multiple vendors; and where each component can be developed and deployed independently.

9.2 Protocols / Standards for Web Service Communication

Naturally, as with any part of computing, there are a smorgasbord of protocols suited for communication: SOAP, XML and WSDL, to name a few (more on these in a bit).

These protocols organise themselves in a stack:

- The protocols at the top of the stack are web service specific protocols such as SOAP, WSDL, UDDI, etc.
- The second level are more general protocols which underpin the top level protocols such as SQL/XML, XML Transformations, XML APIs, etc.
- The third level are definitions of the second layer protocols, such as XML Vocabularies, XML Constructs, etc.
- The bottom level are generalised protocols / standards which are used in web services such as Unicode, URL, HTTP, etc.

9.2.1 Web Service Definition Language

The *Web Service Definition Language* (WSDL) tells the client application what the web service does. It gives the client all the information required to connect to the web service and use all the functionality provided by the web service. The short XML messages describe network services operating messages containing document-oriented or procedure-oriented messages. Web services are defined as a collection of network endpoints or ports; specifying the location of the service using element and description.

The definition of messages (the information passed by the SOAP protocol) is defined in the WSDL protocol. The WSDL document actually tells a client application what are the types of SOAP messages which are sent and accepted by the web service.

WSDL files can be thought of as a postcard, simply containing only the address of the web service and a very short description of the functionality offered by the web service.

9.2.2 Simple Object Access Protocol

The *Simple Object Access Protocol* (SOAP) is a definition of how web services talk to each other, or talk to a client application that invokes them. SOAP is a lightweight (XML-based) protocol for exchange of information in a decentralised distributed environment over HTTP.

SOAP consists of:

- The *SOAP Envelope* that defines a framework for describing what is in a message and how to process it
- A heavy reliance on XML, standards (schemas & name spaces)

SOAP has been designed to be platform and operating system independent. SOAP works on the HTTP protocol which is the default protocol used by all web applications, hence there is no platform specific configuration required to run web services built on the SOAP protocol to work on the internet.

SOAP is the ideal medium to communicate between the web service and client, and as such is recommended by the W3C (the governing body for all web standards).

The SOAP message consists of the *SOAP Envelope* which contains the *SOAP Header* and *SOAP Body*. The purpose of the envelope, header and body is similar to that of an Ethernet Frame with some preamble around source & destination and the payload.

- The SOAP Envelope identifies the XML document as a SOAP message and is used to encapsulate all the details in the SOAP message. It is the root element in the SOAP message.
- The SOAP Header contains *header blocks*, each containing information such as authentication credentials which can be used by the calling application, and definition of complex types that are used in the SOAP body. By default the SOAP messages can contain parameters which could be of simple or complex type
- The SOAP Body contains *body subelements* which contain the call and response information.

This is the actual data which needs to be sent between the web service and the calling application.

9.3 Naming

Naming is crucial within a distributed system. This is so we know exactly what we're talking about when trying to identify resources and ensure efficient access and management. For example, if we referred to our database as 'fred' - this isn't very clear as to what 'fred' is or does. However, calling it 'db-mysql-prd-aws-eu-2' is very clear what it is and where it lives (possibly to its detriment, however).

There are three principle types of name:

- Human Readable Names: URLs, Domain names
- Identifiers: IP addresses, MAC addresses
- Addresses: memory addresses, or server locations (with reference to IP / MAC addresses)

It's important to recognise there may be multiple names pointing to the same resource. For example, a database server might have an IP address, MAC address and one or more DNS names.

Distributed System resources will utilise a naming scheme. This could be a flat scheme, where there is no structure such as that used in peer-to-peer networks; hierarchical naming where names are organised in a tree structure, like DNS; or attribute based naming where names are based on attributes of the resource.

Of course, there are challenges in naming:

- Scalability - handling growing, or shrinking, numbers of names
- Consistency - ensuring names point to correct resources
- Security - preventing unauthorised access and spoofing

9.4 Quality of Service Metrics

Within a distributed system, there are a number of Quality of Service (QoS) metrics which are measured:

- Accessibility - the capability of the system in answering requests
- Availability - is the system there when the user requests it, what is the time to repair the service. This depends on the wider infrastructure, which can pose a challenge
- Scalability - coping with high and low number of user requests in a particular interval
- Interoperability - ability to function with different languages and / or platforms; however this relies on web services to conform to protocols
- Security - the non-repudiation, authentication, authorisation, encryption, traceability and access control of a web service. However this relies on the ability of protocols to support security.

Page 10

Seminar - Web Services & Naming Conventions

📅 2026-03-04

🕒 11:00

👤 Amanda

10.1 SOAP vs. REST

The *Simple Object Access Protocol* (SOAP) uses XML for communication. It requires a supplementary Web Services Description Language (WSDL) to define the XML structure and parameters and works over HTTP, SMTP and TCP. SOAP is slower than REST and is more secure than REST. Commonly SOAP is seen in sensitive industries such as banking.

The *Representational State Transfer* (REST) protocol uses standard HTTP methods (including GET, POST, PUT, and DELETE). The data transferred is formatted in JSON or XML. REST is faster and simpler than SOAP and is used in modern web and mobile applications. An example of REST in action is a weather app retrieving data from a RESTful API.

10.2 Naming

Names facilitate communication and resource sharing. For example, processes need to know the name of the resource which they wish to access.

There are four main categories of names we use in daily computing.

Human Readable:

- URLs
- Email Addresses
- File Names
- User Names

Identifiers (to identify resources):

- IP addresses
- MAC addresses
- Process IDs
- Universally Unique Identifiers (UUID)

Addresses:

- Memory addresses
- Device paths
- Network file paths
- Local file paths

Attribute based names:

- Database tables names
- Tags in cloud storage
- Hashtags in social media

Naming Schemes can come in a few varieties. A *flat* scheme has no structure, such as in P2P networks. A *hierarchical* scheme is organised in a tree structure, such as in DNS.

10.3 Distributed Computing

Distributed Computing is a use of distributed systems where multiple autonomous nodes work together on a single task and communicate via message passing. This is highly scalable where additional nodes can easily be added. This flexibility enables them to handle increased workloads without significant reconfiguration.

The distributed nature of the system enhances fault tolerance; if one node fails - the others can continue to operate which minimises system downtime. Through the distributed paradigm, resources such as CPU, power, memory and storage, can be used efficiently across the network which optimises overall performance. Organisations can also make use of existing hardware and resources instead of investing in expensive supercomputers, or higher power machines, which makes distributed computing a more cost-effective solution.

Distributed Computing has a number of applications:

- Web Services - Many web applications distribute processes across multiple servers to enhance performance and availability
- Online Gaming - Multiplayer online games use distributed computing to synchronise real-time actions between players, providing a smooth experience
- Cloud Computing - Cloud services leverage distributed systems to deliver scalable and reliable resources to users on demand
- Scientific Research - Distributed computing supports complex simulations and data analysis in fields like genomics, physics and climate modelling
- Big Data Processing - Frameworks like Apache Hadoop utilize distributed computing to process large datasets across clusters, improving efficiency and speed

10.3.1 Grid Computing

Grid Computing is a specialised form of distributed computing that connects heterogeneous systems to work on large-scale tasks. This enables organisations to pool resources across various locations, allowing them to process massive amounts of data more efficiently. An example of Grid Computing in action is World Community Grid or Folding@Home.

Grid computing offers reliability with redundant resources, ensuring that the system remains operational even during failures. An example of this would be where a computation task is given to two machines and the results compared to attempt to establish if one is miscalculating the results.

Another benefit of Grid Computing is the fact that it allows different systems and technologies to work together seamlessly, enabling collaboration across various platforms and organisations. By harnessing spare computing resources, grid computing reduces costs associated with acquiring new hardware.

Grid Computing is particularly well suited for handling extensive datasets, making it ideal for scientific simulations and complex analysis.

There are a number of applications of Grid Computing:

- Biomedical Research - Grid Computing enables large-scale simulations in drug discovery and genomics, facilitating faster research outcomes
- Weather Forecasting - Meteorological models utilize grid resources to enhance the accuracy of climate predictions and weather forecasts
- Financial Modelling - Grid systems analyse complex financial data, enabling the simulation of market scenarios and risk assessments
- Engineering Simulations - Engineers use grid computing for extensive simulations and analyses, optimising the design process
- Collaborative Research - Researchers across institutions use grid computing to share data and computational resources, especially useful in collaborative projects.

10.4 Seminar Exercises

Ex. 1: Match the Service Component with its correct description

- Service Provider - Offers the service
- Service Requester - Calls the service
- Service Registry - Lists the service

Ex. 2: Which web service (SOAP or REST) is better for a social media app?

REST. This is because it's lightweight and fast, scalable, easy to integrate and supports multiple platforms.

Ex. 3: Why is Naming important in Distributed Systems

Naming enables resource sharing, simplifies systems management and supports user friendly interaction through the use of meaningful names.

Ex. 4: Match the name to their correct type

- MAC Address - 00:1A:2B:3C:4D:5E
- IP Address - 192.168.1.2
- Universal Unique Identifier - 550e8400-e29b-41d4-a716-446655440000
- Email Address - user@example.com
- Memory address used in RAM - 0x7FFF5A1B3C

Ex. 5: Compare Distributed Computing & Grid Computing

Feature	Distributed Computing	Grid Computing
Architecture	Independent nodes	Networked, heterogeneous resources
Task Management	Centralised or decentralised	Generally decentralised
Resource Ownership	Often owned by a single entity	Resources can be owned by multiple entities
Scalability	Highly scalable with added nodes	Very scalable across networks
Fault Tolerance	Fault tolerance can vary	High fault tolerance due to redundancy
Performance Optimisation	Optimises local resources	Optimises global resources
Use Cases	Real-time applications, gaming etc	Scientific research, simulations, etc
Network dependency	Less dependent on network reliability	Highly dependent on network availability
Complexity	Generally less complex	can be complex due to diverse resources
User Control	More control over resources	Limited control due to resource sharing

Table 10.1: Comparison of Distributed Computing and Grid Computing

Page 11

Lecture - Mobile & Ubiquitous computing

📅 2026-03-10

🕒 09:00

👤 Amanda

Happy Consolidation Week! This lecture is the make-up lecture for Amanda being unable to teach during the first week of TB2.

11.1 Mobile & Ubiquitous Systems

Ubiquitous, meaning to be found everywhere, Computing was introduced to the Palo Alto Research centre in the late 1980s. It was predicted that computational devices would multiply in ‘form and function,’ not just in number, to suit different tasks. Since the 1980s this has happened - with the psychological notion that computers are embedded in everyday life.

Ubiquitous computing is the concept of integrating computing capabilities into everyday objects and environments, making technology seamlessly available and accessible anytime anywhere, and without requiring user interaction. A typical example of this is Internet of Things (IoT) devices interacting with more traditional computational devices, often providing realtime information to users without their demand. A use-case of this context-based action may be that a user enters a room wearing an active badge; which is detected by infrared sensors and then a display responds to the user’s presence to display custom content etc.

Wearable computing is where users carry wearable computing devices on their person, either attached to or within the fabric of their clothes, or worn like jewellery. This is unlike the handheld devices as wearable technology often operates without users directly interacting with them. One of the most common examples of wearable computers is smartwatches, but other examples include glasses, jewellery or chips embedded in clothing. Often they are used for data collection, are read only and are programmed to specific requirements - so are not general purpose devices.

A Smart Space is a physical place with embedded services, meaning any services provided which is only or principally provided within that physical space. Typically smart spaces contain relatively stable computing infrastructure, which may include conventional servers, end user devices (printers, laptops, etc) as well as wireless networking infrastructure. Smart spaces act as environments for devices that visit and leave them - users can bring in and depart with devices they carry or wear; this is both in the physical and logical domains wherein a user may physically enter a smart space with their smartphone which would cause the device to move logically between command components within the space.

Devices are limited in energy and processing power. We have an expectation of devices that they are able to interface with the world, but this presents challenges: computation power, memory and storage limits, and network bandwidth. Increasing the capabilities of any of these presents an increase in energy consumption, which itself is a challenge as there is a trade-off between battery power for devices where it may be limited or run out, and mains-powering devices where the portability of them is considerably reduced.

Due to the highly dynamic and unpredictable changes in the set of users, hardware and software in ubiquitous computing - much of the networking is described as spontaneous networking. This presents a level of volatility in the networking including failures of devices and communication links, power

failure for power-dependent devices, changes in the network characteristics (i.e. bandwidth), and creation and deletion of associations between software components residing on the device.

A key component of ubiquitous systems is the ability to interact with the real physical world. This takes place through sensors measuring physical parameters, such as orientation, load, light, or sounds. This data is then computed by a computational unit. Directly or indirectly - an actuator, software controllable device, may be instructed to make a change to the air conditioning, smart home devices, motors, etc.

Distributed Systems are based on the trust of sharing information, functionality, etc, with multiple different systems. The public perception of sensing devices is to not trust them - due to privacy concerns of tracking the users, discovering routines of the uses, impacting online purchases, etc.

11.2 Association Problems & Discovery Services

Volatile Components need to interoperate. Their access to the network is essential for communication. Components either associate to the services in their smart space and/or provide services elsewhere. This presents a challenge of scale and scope, in that there may end up being a large number of devices and software components in a single space. Associates are not constrained by boundaries - they can use discovery services to aid the process.

There are two types of association - pre-configured and spontaneous. Pre-configured association is commonly service driven, such as the association between an email client and server. Spontaneous association may be human driven, as in the association between web browsers and web servers; data-driven as in the association between P2P file sharing applications; or physically driven as in the association between mobile and ubiquitous systems.

Within smart spaces - it is common for there to be a server providing discovery services, this provides a service lookup for devices & services which can register and de-register. There are, however, a number of challenges with the implementation of such as a service - in that the client needs are determined at runtime.

There exist two models for getting the information to clients about what services exist. The Push Model advertises services through multicast which client devices pick up and cache, they can then query their own caches when they need to discover a service; this does present a challenge that the service update multicasts need to be transmitted at a reasonable frequency. Alternatively, the Pull Model is where client devices transmit multicast messages requesting services, and the devices providing those services respond.

There are two implementations of discovery services. The server based implementation uses a centralised directory, similar to the pull model discussed above. While the serverless implementation (decentralised P2P architecture) does not contain a central directory, rather each device maintains their own directory in their own cache. Both of these implementations have challenges around clients managing volatile services.

There are, of course, challenged with discovery services:

- Bandwidth usage
- Push model advertising is wasteful of network resources
- Pull model is more efficient - but may need more responses
- Management of volatile services: clients lease a service for a length of time so the clients need to renew the lease for continued use as if the service is not renewed it is released
- Scale
- Defining and enforcing the boundaries of smart spaces

Of course, it is possible to get the human to do the association. The user can select the scope identifier (i.e. room number), for example. It is also possible to use sensing and physically constrained channels to keep discovery within a manageable scope; such as reading the code for a smart space identifier or using an infrared transmitter to propagate the space identifier. Direct association, using address sensing or physical correlation is also an option.

11.3 Programming Models for Volatile Systems

There must be a standardised interface between devices and services, to ensure interoperability. This relies on the use of protocols and programming models being followed, with the aim of avoiding the ‘lost opportunity problem.’ However this presents some challenges - to improve software interfaces compatibility; interfaces should be heterogeneous; and for every N interfaces, N^2 adapters are needed to be determined at runtime.

Data oriented programming is an unvarying service interface, for example Unix pipes. This is used in event driven systems wherein the event service itself provides the interfaces for publishers to publish and subscribers to consume events; naturally there are challenges with discovery and agreement on the event service instance & service scope. The Tuple Space model allows exchange of application specific structured between different components.

To ensure interoperability between different services - interfaces are constrained to be of one (or two) types which are globally recognised as standards. If smart spaces had their own programming interface, this would inhibit mobility. Components moving in the smart space needs to communicate via adapting to interoperate with services, which requires complex runtime support. Alternatively - some services are self-describing via data oriented systems using XML or JSON.

11.4 Sensing & Context Awareness

There are a number of challenges with context aware systems:

- Integration of sensors - specialised knowledge may be required
- Abstraction from sensor data - applications will need to abstract from raw sensor data (e.g. location)
- Sensor fusion - multiple sensors data need to be combined and interpreted to perform a complex sensing task such as detecting the presence of a human
- Context is dynamic - applications typically need to respond to changes in context

There are a number of different architectures for software used in wireless sensor networks:

- In-network processing - processing on the sensor nodes or aggregating from nearby nodes
- Disruption tolerant networking - no end-to-end exists continuously, so communication is handled opportunistically
- Data oriented programming - directed diffusion takes place where tasks are injected into the system at sinks (specific nodes) and then runtime system diffuses the tasks in the system; or through data query processing

Page 12

Lecture - Fault Tolerance

📅 2026-03-17

🕒 09:00

👤 Amanda

This is the final lecture with new content before the exam next week. This exam will cover everything up to this point, and after that we will be done with Distributed Systems (the ‘and Security’ still to come after Easter...).

This lecture won’t paint the full picture of Fault Tolerance, the seminar will fill in the blanks.

12.1 Introduction to Fault Tolerance

Definitions

Fault Tolerance The system’s ability to continue operating correctly despite failures.

Fault Tolerance isn’t so much about eliminating failures, this is computing after all so we can never achieve that, but more looking at how we can handle the failures efficiently to maintain system reliability. Generally it’s aimed for systems to have a 99.999% uptime. At first this may seem like a weird number, given 100% is so close, but it represents the reality that there will be some amount of downtime, either for scheduled activities or for unplanned outages.

Distributed Systems consist of multiple nodes and networks where failures are inevitable due to hardware issues, software bugs or network disruptions. The swan analogy is relevant here - in that under the surface the different nodes may be working hard to maintain the reliability and availability expected of the distributed system but above the surface (where the users see it) - the system is operating as expected with no degradation.

Fault Tolerance isn’t just about the ongoing use of a system. It has to be factored in at the design and implementation stages as well. For example considering any external influences on the system, such as an us-east-1 outage or power failure in an on-prem data centre.

12.2 Failures Within Distributed Systems

There are a number of types of failure within distributed systems. This is not an exhaustive list of them. It’s important to recognise that there are more types of failure than “it crashed” or “I can’t access it.”

A *crash failure* is where a completely stops responding, for example due to a power failure or process crashing. This halt in responding could be for a long time (until the power is restored or an operator fingers the power button), or for a shorter ‘blip’. This can affect data integrity - such as if a database server crashes mid transaction.

The *omission failures* are failures which happen when messages are lost or dropped during transmission. For example, network congestion may lead to requests to an API being dropped.

Timing failures are categorised as when a system responds too early or too late. This is highly problematic in time-sensitive applications such as financial transactions; where if a banking system

fails to process transactions in real-time there may be a backlog building up which can lead to incorrect data being processed.

A *Byzantine Failure* is a failure which has come about as a result of a node behaving incorrectly or maliciously (such as sending false data). This can cause problems in distributed systems where the other nodes do not know that a node has failed and so believe that the data it is producing is the truth, such as a hacked node in blockchain broadcasting fake transactions.

Network Partitioning failures are failures related to the network, not necessarily a component of the distributed system itself. This could be caused by a node being isolated due to network issues such as a cloud region disconnecting from a global system.

Failures are detected, and sometimes compensated for, using a number of methods:

- Heartbeat Mechanisms - nodes send periodic 'heartbeat' signals and the lack of these suggests failure
- Timeout-Based detection - if no response is received within a time limit, the node is marked as failed
- Gossip Protocols - Nodes spread failure information throughout the system

Some failures are pre-empted and auto recovered. For example Amazon EC2 Compute Instances have the ability to auto-scale so by detecting failing servers they can spin up replacements automatically.

12.3 Redundancy, Replication & Recovery

Generally, as we have seen above, a failure is going to lead to some form of data loss. This loss of data is problematic - so we look to minimise the quantity of problem by replicating the data across multiple nodes, ideally across multiple geographical regions. There are a number of different replication / redundancy models:

- Data Replication - where one server acts as the primary with additional servers holding a replicated copy of the primary. This is the most commonly used backup model for servers.
- Quorum-based replication - where the majority of the nodes have to agree before an update is made to the replicated data
- Process Redundancy - has two modes; Active Replication is where all nodes run in parallel, while Passive Replication is where one node is the primary node with the others acting as backups
- Voting Mechanisms - these are used in databases to ensure consensus before committing transactions to the database

A common example of data replication is that of Google Drive where it stores multiple copies of the data in different locations around the globe.

After the failure has occurred, we may go to our replicated data or redundancy server and realise it's also failed. When this happens the next logical option is to 'roll-back' the system to an earlier state. This is achieved through checkpoints.

Checkpoints are periodic snapshots of the systems entire state - similar to a full hard drive backup. A checkpoint is restored through *rolling back* which restores the system state from the checkpoint. Note that this will lose any data between the when the checkpoint was taken and when the system crashed. With checkpoints and other full system backups - there is a need to validate that the checkpoint has saved properly commonly through testing, rather than just assuming it has then having a nasty discovery when a rollback is attempted.

Log messages can provide valuable insight into what the system was doing when it crashed. These can be examined to understand what happened and what may have lead to the system failing in an

attempt to prevent it happening again.

Self Healing Systems are systems which use machine learning to monitor key variables and learn what happens when the data indicates the system is going into a risk of failure then resolving it before the crash happens.

12.4 Consensus Algorithms

As we have seen above - there is often some degree of consensus required for making a decision about actioning fault recovery. This begs the question “How do distributed nodes agree on a single value, even with failures?”

Unlike humans, nodes can't sit around in a circle and raise their hand at the appropriate option in consensus based decision making - that would be a bit strange. So instead there are algorithms used:

- The *Paxos Algorithm*, developed by Lamport in 1998, ensures agreement in unreliable networks; however does not handle Byzantine Failure.
- The *Raft Algorithm*, developed by Ongaro & Ousterhout in 2013, is used in Kubernetes for leader election; however does not handle Byzantine Failure.
- The *Byzantine Fault Tolerance (BFT)*, developed by Liskov and Castros, is used in blockchains to handle malicious nodes.

12.5 Fog & Edge Computing

This is a departure from the Fault Tolerance we've been discussing so far in this lecture. The topics discussed here very much harp back to last weeks Ubiquitous Computing concepts.

Definitions

Fog Computing resources are placed closer to the network edge, often in local gateways or routers. This reduces latency and handles data near the source.

Edge Processing happens directly on the edge devices (such as IoT Sensors, cameras, phones, microcontrollers, etc). This is about as close to the data source as it's feasible to get.

Edge Computing handles device level decisions. They process the data locally so they can react immediately. Edge devices are useful because of their ultra-low latency, immediate reaction and the fact that there is no need to send everything through the network. For example:

- Traffic sensor detects a pedestrian and immediately triggers a flashing warning
- Camera identifies a car running a red light and logs the event
- Pollution sensor measures air quality and alerts if the levels spike

Fog Computing coordinates data across a wider area. Fog nodes sit between the edge devices and the cloud. They collect data from multiple sensors across a neighbourhood, junction or district. Fog nodes have more power and can provide localised analytics without having to transmit all the data to the cloud.

This concept can be thought of as the edge computing makes immediate, local decisions based on the single or very small number of sensors it is connected to. The data from edge nodes is aggregated to fog nodes who will aggregate multiple edge nodes data together. Fog nodes provide regional coordination across their network. Fog nodes then aggregate together to provide information to the cloud which handles the longer-term strategy and storage of the data gathered.

In a scenario, the edge, fog and cloud paradigms can be applied as follows:

- A roadside camera detects heavy traffic - edge node
- The edge node sends processed data, such as 'traffic density high' to a nearby fog node
- The fog node analyses data from all junctions in the district
- The fog node adjusts traffic light cycles to reduce congestion
- The fog node sends summaries to the cloud for long-term planning

Page 13

Seminar - Fault Tolerance

📅 2026-03-18

🕒 11:00

👤 Amanda

In the exam, next week, there will be between 15 and 20 Multiple Choice Questions. They do not use negative marking and there will be 5 answers to choose from, it should be reasonably easy to pick out the right answer. The MCQs alone should be enough to get 40% overall in the exam.

After the MCQs there will be two scenario based questions. These will be similar to what we've experienced in the Seminars and QotW but will draw in knowledge from the last 8 weeks, not just 1 week. The scenario questions will be composed of different sub-questions with some being dropdown and some being unlimited-sized free text boxes. If the dropdown option is selected incorrectly but a sufficient justification is put in the free text box following it - it's possible to get some marks still. We don't need to write lengthy essays - bullet points will suffice.

Next weeks's Seminar & Lecture will take place as usual - although the Seminar will be literal hours before the exam...

13.1 Replication

Definitions

Active Replication All nodes run in parallel, with data instantaneously being replicated to the node / process

Passive Replication One node runs as the primary, with others running as backups - data not instantaneously replicated to the backups

The trade-offs between active and passive replication in process redundancy are based on: performance, fault tolerance, and consistency. Active replication is used when strong consistency and fast failover is needed such as within banking or real-time trading systems. Passive replication is used when performance and resource efficiency are required, such as in cloud services or web applications.

Often in a real-world scenario there would be a hybrid deployment with some active replication which replicates to some passive replication. This hybrid option gives the benefits of both systems while saving some money.

13.2 Consensus Algorithms

Further to the discussion in the lecture around consensus algorithms - we'll now see how they work in detail.

13.2.1 Paxos Algorithm

The *Paxos Algorithm* aims to ensure consensus in unreliable networks where nodes may crash or messages may be lost. It works using a three-phase protocol which involves three different types of nodes (Proposers, Acceptors and Learners):

1. **Prepare Phase:** A proposer sends a proposal with a unique number to the acceptors. If the acceptors haven't already accepted a higher-numbered proposal, they respond with the last accepted value.
2. **Accept Phase:** The proposer selects the most recently accepted value and sends an accept request. Acceptors acknowledge it if no higher proposal exists.
3. **Learn Phase:** If a majority of Acceptors accept a proposal, it becomes the agreed value. The decision is sent to Learners who apply the decision.

Paxos Algorithm is used where durability is needed to replicate large datasets such as a file or database and works in asynchronous networks. Paxos selects a single value from one or more of the values that are proposed and then broadcasts that value to all of the cooperating computers.

After it has run - all of the nodes agree upon the proposed value and the cluster clocks forward. It is possible for a single node to be all three roles.

13.2.2 Raft Algorithm

The *Raft Algorithm* improves over the Paxos Algorithm. There are still three types of node involved:

- **Leader** - Only the server elected leader can interact with the client. All other servers sync themselves with the leader. At any given point in time there can either be 1 or 0 leaders. Throughout the lifespan of a distributed system, it is possible for the leader role to change between nodes.
- **Follower** - These servers sync their copy of the data with the leader at regular time intervals. If the leader server goes down, one of the followers can contest an election and become the leader.
- **Candidate** - At the time of contesting for leader status, an election to become the leader server takes place. Each server can ask other servers for votes (hence where the name Candidate comes from). Initially all servers are in the candidate state. The candidate with the most votes will become the new leader server.

Using the Raft Algorithm, the Leader ensures that all servers maintain the same sequence of data with the following steps:

1. Client sends a request to the leader
2. Leader appends the request to its log and sends an AppendEntries message to its followers
3. Followers copy the entry and send an acknowledgement back to the leader
4. When a majority acknowledges, the leader commits the entry and applies it to the system
5. The leader notifies the client of the successful operation

13.2.3 Byzantine Fault Tolerance

Byzantine Fault Tolerance (BFT) is used with nodes exhibit arbitrary or malicious behaviour. It employs cryptographic methods, redundancy and quorum systems to ensure the integrity and consistency of the system despite the presence of faulty nodes. In the event of nodes failures or malicious behaviour, the system can identify and isolate the faulty nodes while continuing to operate normally.

A byzantine fault is a type of failure where a node: crashes or stops responding; sends conflicting or incorrect information to different parts of the system due to corruption, hacking or intentional attacks; or acts maliciously to disrupt the system (e.g., lying about the state of data). BFT Is named after the Byzantine General's Problem which describes the difficulty of achieving consensus when some participants may be unreliable.

A BFT system operates under the assumption that at most $\frac{n-1}{3}$ nodes can be faulty, when n is the total number of nodes in the system. This means for the system to function correctly:

- At least $3f + 1$ nodes are needed to tolerate f Byzantine failures
- At least $2f + 1$ honest nodes must reach consensus to outvote malicious nodes

To achieve BFT, the system will follow these steps:

1. Nodes exchange messages to verify information from others
2. Majority agreement (consensus) is required before a decision is finalised
3. Verification ensures that faulty or malicious nodes cannot influence the system

There are three different types of BFT algorithms:

- **Practical BFT** - uses a leader-based approach to reach consensus among nodes. The leader isolates a node if they begin to misbehave.
- **Federated BFT** - used in decentralised networks where each participant chooses whom to trust (or not). If a node's trusted comrade begins misbehaving - they get isolated.
- **Proof of Work** - used in Bitcoin and other blockchains.

BFT can be seen in a number of different applications; it tends to be seen predominantly in financial or critical systems where a fault presents significant issues:

- **Blockchain Networks:** BFT is crucial in permissioned blockchains such as Hyperledger Fabric and PBFT-based consensus models to prevent malicious actors from tampering with transactions.
- **Aerospace and Autonomous Systems:** Spacecraft, satellites and self-driving cars use BFT to handle faults or cyber attacks in mission-critical environments.
- **Financial Systems:** Distributed banking and payment networks required BFT to prevent fraudulent transactions and ensure ledger integrity.
- **Critical Infrastructure:** Smart grids and Industrial Control Systems (ICS) use BFT to prevent system failures due to malicious nodes or cyber threats.

13.3 Seminar Exercises

Ex. 1: What fault tolerance mechanisms would be executed when a server crashes in an e-commerce platform?

- Detection - through a server heartbeat or timeout
- Automated reporting in a timely manner to surrounding servers through the gossip protocol
- Automatic failover to redundancy server
- Possibly some amount of self-healing even after the server has gone down
- Alert a human to investigate / resolve
- Failure must be transparent to the end-user

Ex. 2: What failure type would buffering issues fall under if you were managing a video streaming service?

- Video streaming service would use UDP, so there is some level of fault built into the protocol (i.e. dropped packets are handled without retransmission)

- Could be Timing Failure (UDP doesn't reorder packets on receipt so if packets are received in the wrong order, there's a risk of them being displayed in the wrong order) or Crash Failure
- There is a chance Network Partitioning could play into it - if a change is made to the network which restricts access to some component of the video streaming platform, however this is less likely than Timing or Crash failures.

Ex. 3: What are the trade-offs between active and passive replication?

Criteria	Active Replication	Passive Replication
Performance	Higher overheads (request by replicas)	Lower overheads
Consistency	Strong (depends on synchronisation accuracy)	Potential data loss (if primary fails)
Speed (failover)	Fast (any replicas can takeover)	Slower (backup must detect failure)
Complexity	More complex (needs consensus algorithms)	Simpler (primary to back up)
Scalability	Limited due to synchronisation overhead	Scalable for heavy write systems

Table 13.1: Comparison of Active and Passive Replication

Ex. 4: Compare the Raft and Paxos Algorithms

Feature	Raft	Paxos
Understanding	Simple, well-structured	Complex to implement
Leader Elections	Explicit Leader	Implicit Leader
Log Structure	Sequential log replication	Flexible but complex
Performance	Efficient with clear roles	Complexity causes a slower performance overall

Table 13.2: Comparison of Raft and Paxos Algorithms

Ex. 5: A banking system crashes in the middle of a transaction. How does checkpointing help in recovery?

- A checkpoint taken before starting the transaction gives the state of the system before the transaction begins. Then if something goes wrong with the transaction (such as money not leaving the sender, or arriving at the receiver) - the checkpoint can be rolled back to ensure that there are no double-sends or receives of the money.
- Multiple checkpoints can be used throughout the transaction - to allow rolling back of a single stage rather than just the transaction as a whole; for example 'has the money left the sender's account?' or 'has the money arrived at the receiver's account?'

Part II

Security

Page 14

Lecture - Foundations of Distributed Systems Security

📅 2026-04-21

🕒 09:00

👤 Fahad

Today marks the beginning of the second part of this two-part module. *Fahad Ahmad* will take us through four lectures focusing on the security of Distributed Systems. The Seminars from before Easter will become Tutorials in a computer lab. These four weeks will be assessed during the May assessment period in a single CBT with 24 Multi-Choice Questions and 1 long question consisting of three parts with an equal weighting between MCQs and long. A mock exam will be released during week 4.

14.1 Security: What Is It?

Information Security is the preservation of confidentiality, integrity and availability of information (the CIA triad we've seen before).

Definitions

Confidentiality The property that information is not disclosed to unauthorised individuals, entities or processes

Integrity The property of safeguarding the accuracy and completeness of assets

Availability The property of being accessible and usable upon demand by authorised entity

There are also a number of other essential definitions we will need for this module.

Definitions

Authentication Provision of assurance that a claimed characteristic of an entity is correct

Threat A potential cause of an unwanted incident, which can result in harm to a system or organisation

Vulnerability A weakness of an asset or control that can be exploited by one or more threats

Impact The result of an information security incident, caused by a threat, which affects assets

Risk Expressed in terms of a combination of the consequences of an event and the associated likelihood of occurrence

Control Any process, policy, device, practice, or other action which can modify networks

Security has always been needed in distributed systems and computer networks. Historically a network would have been a private entity, where all connected devices either are hard-wired to the same central point, or use local wireless connections. This was reasonably easy to secure as there are minimal points of data entry & exit, and these often would go through enterprise hardware which supports security.

In the more-recent times, teleworking has become fashionable. This is where one or more users requires

remote access to the corporate network, which has to be provided in a safe and secure manner. This principle can be extended to see that remote sites have to be connected to the primary corporate site; again in a safe and secure manner.

The more devices we want to connect to our network, the bigger risks we face. The more devices, connections, interfaces, etc, provide a greater surface area for attacks to be launched on; that is each external-facing connection, web-service, etc will come with it's own set of vulnerabilities which have to be managed.

There are three new problems introduced with opening the network:

- How do I know who I'm talking to?
- How can I ensure nobody else hears our conversation?
- How can I be sure that what I've said is heard (correctly)?

Returning back to the CIA triangle, this is pretty simple. We can apply: Authentication to know who we're talking to; Confidentiality to prevent conversations being listened in on; and Integrity to ensure that the message is received correctly.

There are a number of definitions related to threats and attacks which will come up in this module.

Definitions

Leakage Acquisition of information by unauthorised recipients

Tampering Unauthorised alteration of information

Vandalism Interference with the proper operation of a system without gain to the perpetrator

Eavesdropping Obtaining copies of messages without authority

Masquerading Sending or receiving messages using the identity of another principal without their authority

Message Tampering Intercepting messages and altering their contents before passing them onto the intended recipient

Replaying Storing intercepted messages and sending them at a later date

Denial of Service Flooding a channel or other resource with messages in order to deny access for others

Obviously this is bad, and we need to solve this. To do so, we introduce the concept of *Cryptography*.

14.2 Cryptography

Cryptography is a technical control used to reduce information security risks. It turns plaintext (the readable version of the message) into ciphertext (the unreadable version of the message) and then back again, on demand. There are five elements to cryptography:

- The ciphertext: c
- The encryption algorithm E
- The plaintext: m
- The key K (sometimes represented differently depending on the type of key, more on this later)
- The decryption algorithm: D

Encryption links four of these together; where the ciphertext is constructed by applying the encryption algorithm to encipher the plaintext using the key. Or mathematically:

$$E_k(m) = c \quad \text{or} \quad E(k, m) = c \quad \text{or even} \quad c = \{m\}_k$$

Decryption reverses the process; where the plaintext is recovered by applying the decryption algorithm to decipher the plaintext using the appropriate key. Or mathematically:

$$D_{k'}(c) = m \quad \text{or} \quad D(k', c) = m$$

Modern cryptographic algorithms abide by the following principles:

- They have a large enough key space to resist exhaustive searches
- They are resistant to frequency analysis
- A small change in the plaintext should result in a large change in the ciphertext
- Their security depends only on the secrecy of the key, and not on the secrecy of the algorithm

There are two broad forms of cryptographic algorithms used.

14.2.1 Symmetric Encryption

Symmetric encryption uses the same secret key to encipher and decipher the message (meaning $k = k'$). The encryption methods can be extremely efficient as they require minimal processing, however both the sender and receiver must possess the same encryption key, which means that if either copy of the key is compromised, an intermeidary can decrypt and read messages.

It would be great if we can just use pre-shared symmetric keys with everyone that we need to communicate with. However this presents a significant problem, in that for 2 or 3 communicators, the number of keys required is low; however this number grows quickly. Where there are 5 people communicating, 10 keys needed; 6 communicating, 15 keys needed; 7 communicating, 21 keys; or 8 people communicating, means 28 keys are needed. The number of keys needed can be calculated using $n \times (n - 1)/2$ where n is the number of people trying to communicate.

Common symmetric algorithms are: DES, IDEA, RC4, AES.

14.2.1.1 XOR Cipher

At it's simplest, a symmetric algorithm may simply XOR a binary sequence (key) with the binary representation of the message wishing to be transmitted. Then the key and received message can be XOR'd together to result in the originating message.

14.2.1.2 AES Encryption

Symmetric Encryption can also exist in a more complex algorithm, for example AES. AES works in a series of rounds.

In the first round, the entire cipher key is XOR'd with the input in a step called AddRoundKey. The output from this is fed into the main rounds.

Each of the 9 main rounds consist of the same four steps:

1. SubBytes
2. ShiftRows
3. MixColumns

4. AddRoundKey - except this time, a round-specific key is used which is derived from the main cipher key

The output from the 9th main round is fed into the output of the final round.

The final round completes three steps to produce the final output from the AES algorithm:

1. SubBytes
2. ShiftRows
3. AddRoundKey - again, this is a round-specific key derived from the cipher key

14.2.2 Asymmetric Encryption

Asymmetric encryption (sometimes known as public-key encryption) enciphers and deciphers using two different, but related keys (meaning $k \neq k'$). If key k encrypts the message, only the key k' can decrypt it. Generally one key will serve as a public key, which is known to all who wish to communicate with the sender, and one serve as a private key, which is known only to the sender.

Common asymmetric algorithms are: RSA and ECC.

14.2.2.1 Textbook RSA

The ‘Textbook’ RSA algorithm is a textbook-suitable version of the RSA algorithm. It has five steps:

- Choose two large primes p and q , and calculate $n = p \times q$
- From n , follow some maths steps to calculate the value e and d
- Publish n and e , keep d a secret, and destroy p & q
- Encryption of m is now $c = m^e(\text{mod } n)$
- Decryption of c is then $m = c^d(\text{mod } n)$

Example: Textbook RSA

If we take the string ‘DSS’ that we want to encrypt, we can first turn it into ASCII: ‘86 83 83’

We can now encrypt them in turn, knowing $e = 131$, and $n = 323$:

- $68^{131}(\text{mod } 232) = 102$
- $83^{131}(\text{mod } 323) = 315$

This means our transmitted message is ‘102 315 315’.

On the receiver’s end, we receive the message (‘102 315 315’). We also know that $d = 11$ and $n = 323$, which means we can decrypt as follows:

- $102^{11}(\text{mod } 323) = 68 = D$
- $315^{11}(\text{mod } 323) = 83 = S$

From this we can reconstruct our string ‘DSS’.

14.3 Digital Signatures

Asymmetric cryptography provides a built-in technique to sign messages with non-repudiation. If we take the scenario that Alice encrypts a message with her private key $E_{sk_A}(m)$ and then anyone who receives this can decrypt using pk_A which everyone has. Everyone knows that only Alice could have

created the ciphertext as that requires the use of sk_A which Alice is the only person to have.

Digital Signatures are an extension of the above - the idea that we can tack something onto a digital message to validate it's authentic, unforgeable and non-repudiable.

Definitions

Authentic It convinces the recipient that the signer deliberately signed the document and it has not been altered by anyone else

Unforgeable It provides proof that the signer, and no one else, deliberately signed the document. The signature cannot be copied and placed on another document

Non-repudiable The signer cannot credibly deny that the document was signed by them

This brings us onto Digital Signatures with secret / symmetric key. This is considerably simpler than asymmetric as A & B both use k_{AB} to communicate and they are the only ones to have this key. Therefore no other entity cannot claim to have created the ciphertext received by B. The issue of key exchange, distribution and management remains an issue however.

14.3.1 Hash Functions

A *hash* function is an algorithm which produces a fixed-length bit pattern that characterises an arbitrary-length message. They are another important cryptographic primitive. Popular hash algorithms include SHA-1, MD5 and SHA-2.

They are also known as one-way functions, or trapdoor functions, that should satisfy the following conditions:

- Given M , it is easy to compute $H(M)$
- Given $H(M)$, it is hard to compute M
- Given M , it is hard to find another message M' , such that $H(M) = H(M')$
- it is hard to find two messages M and M' such that $H(M) = H(M')$

The last two look similar, however they're different.



There's an explanation on the slides as to why they're different to do with birthdays. This wasn't discussed in the lecture.

In reality, a digital signature of a document is generated by using a hash function to generate the document's hash then encrypting that with the senders private key. On the receiver's end - they can use the senders public key to decrypt the hash and compare this with their own hash generated from the received document. If the two hashes match then the document hasn't been tampered with.

14.3.2 Signing a Long Message

A hash function can be used to efficiently sign a long message. This is done by calculating the hash of the message (using a predetermined hash function) then encrypt the hash with the private key: $m.E_{sk(a)}(H(m))$

Then anyone who receives this can verify that it was sent by entity 'a':

1. decrypt using $pk(a)$ to calculate $H(m)$
2. also calculate $H(m)$ directly by hashing the message
3. check that the two values match, if they don't - reject the message

14.4 Digital Certificates

A *digital certificate* binds a subject to a private key. This contains a digital signature which is used to verify the issuer of the certificate, called the ‘Certificate Authority’ (CA).

A certificate has a *certification path* which is the chain of certificates between any given certificate and its trust anchor (CA). Each certificate in the chain must be verifiable in order to validate the certificate at the end of the path. The certification path may be long or short. For example:

1. Trust Anchor
2. Intermediate CA
3. End-entity Certificate

Alternatively, the certification path may be longer:

1. Trust Anchor
2. Intermediate CA
3. Issuing CA
4. Subject CA
5. End-entity certificate

Both of these work in the same way that when a resource is attempted to be accessed, its root certificate is verified. For example, accessing `google.com` through a web browser: the web browser receives the certificate for `google.com` and verifies that its trust anchor is in date, not revoked, and trusted by the web browser. This is the same for other uses of certificates, just that the trust list may be in a different place.

There are approximately 60 Root CAs in the world, which all public certificates are derived from. An enterprise may also have their own Root CA and therefore their own PKI infrastructure which is used to manage internal trust to internal resources. Obviously in the later scenario, any devices which needs to trust the resources needs to trust the organisations Root CA.

A Public Key Infrastructure (PKI) is a framework that manages digital certificates and public-key encryption. It addresses security requirements, including authentication via a framework for public key certificates. The framework specifies the information objects including data types for a PKI including:

- public-key certificates
- certificate revocation lists (CRLs)
- trust broker
- authorization and validation lists (AVLs)

The framework also addresses issuing, managing, using and revoking certificates.

Page 15

Lecture - Network Layer Security

📅 2026-04-28

🕒 09:00

👤 Fahad

Some content in this lecture overlaps significantly with an *IT & Internetworking Security* lecture from a few weeks ago.

15.1 IPsec

The *Internet Protocol security* framework is a security architecture for IP. There are three key components, some of which each have options for implementations, these are used in combination to build up the framework:

- Security Protocol: Either Authentication Header (AH) or Encapsulating Security Payload (ESP)
- Key Management: manual or automated (using the Internet Key Exchange)
- Cryptographic Algorithms, for authentication and encryption

As we can predict, IPsec provides confidentiality, integrity, authentication and anti-replay mechanisms. *Confidentiality* is provided through encryption of data meaning only the sender and receiver can read the data. *Integrity* is provided by calculating and verifying hashes to enable the receiver to check the packet was not modified in transit. *Authentication* is enabled through ensuring that the sender and receiver authenticate each other to make sure that they are communicating with the intended entity. *Anti-replay* prevents duplicate packet transmission, and is supported in IPsec through assigning each packet a sequence number.

The cryptographic algorithms, as mentioned above, are not explicitly stated in the IPsec framework. Rather the framework leaves space for a modular system of protocols to be inserted depending on the requirements of the network. However, the selected algorithms must come from a list of approved algorithms such that they work with AH and ESP; similarly for IKEv2.

Due to each component of IPsec being specified under separate intertwined specifications, a single component can be updated without the entire specification having to be re-written.

IPsec creates a boundary between unprotected and protected interfaces for a host or a network. For example on a edge router between a LAN and the Internet. Then when traffic wants to cross this boundary, one of three things happens:

- traffic passes unimpeded
- traffic is afforded security services via AH or ESP and sent on its merry way
- traffic is discarded

IPsec is used to protect traffic in numerous different scenarios:

- between a pair of hosts
- between a pair of security gateways - as in a site-to-site connection over the internet
- between a security gateway and a host - as in a teleworker connecting to the head office

15.1.1 Security Protocols

Authentication Header (AH) provides data integrity, authentication of IP packets, and protection against replay attacks. The first two are achieved through the use of a Message Authentication Code (MAC), i.e. HMAC-SHA1-96. IP packets expand when AH added with item such as:

- next header - type of the header following this header
- payload length - length of AH
- Security Parameter Index (SPI) - identifies a SA
- sequence number
- authentication data

Encapsulating Security Protocol (ESP) provides the functionality of AH, plus confidentiality. When applied to an IP packet, ESP adds an ESP header, encrypts the payload, and adds an ESP trailer. An ESP packet contains:

- security parameter index (SPI)
- sequence number
- payload data (encrypted)
- padding - to achieve data length that is a multiple of 32 bits (encrypted)
- padding length (encrypted)
- next header (encrypted)
- (optional) authentication header

15.1.2 Modes of Operation

There are two modes of operation: *transport mode* and *tunnel mode*. A host implementation of IPsec must support both transport and tunnel modes; while a security gateway implementation must support tunnel mode and may support transport mode.

Transport mode is the least secure of the two. Only the payload of the IP packet is always protected; with the header not encrypted in ESP, but some components of it encrypted in AH. The data is protected from source to destination, but the header information is transmitted as cleartext. This is used predominantly between hosts.

Tunnel Mode is the more secure of the options. The entire IP packet is encrypted (this meaning both the IP header and payload), and this becomes the payload of a new IP packet. The new (outer) IP packet may contain different source and destination addresses, obfuscating the original source and destination data. This provides data flow confidentiality to some extent and can be used in gateway-gateway, host-host, or host-gateway paradigms.

15.1.3 Methods of IPsec

There are four combinations of security protocol and operating mode. These all come in context of the original IP packet, as below.

IP Header	TCP Header	Data
-----------	------------	------

Figure 15.1: Original IP Packet

15.1.3.1 Authentication Header: Transport Mode

In transport mode, while using AH, the HMAC is computed over:

- immutable header fields (the fields that do not change in transit) which include source address, IP header length
- the AH (except authentication data field)
- IP data

The values for the excluded fields are set to zero during HMAC computation.

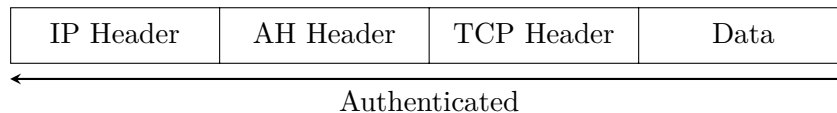


Figure 15.2: AH Transport Mode

15.1.3.2 Authentication Header: Tunnel Mode

In tunnel mode while using AH, the HMAC is computed over:

- immutable header fields (fields that do not change in transit) which include source address, IP header length
- the AH (except authentication data fields)
- IP data

The values for excluded fields are set to zero during HMAC computation.

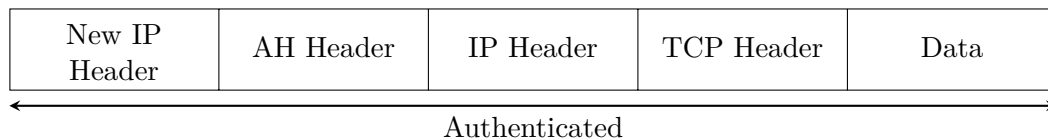


Figure 15.3: AH Tunnel Mode

15.1.3.3 Encapsulating Security Payload: Transport Mode

Authentication is only provided over ESP header and data fields. Only the data is encrypted, meaning the source and destination are clear.

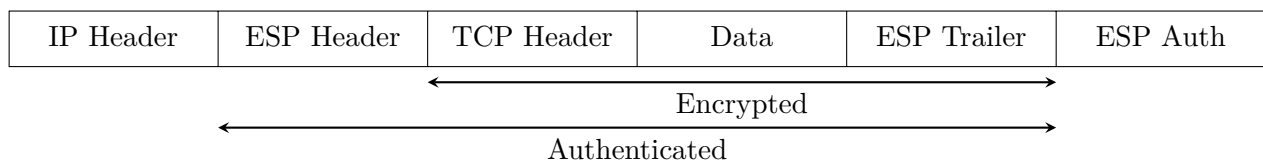


Figure 15.4: ESP Transport Mode

15.1.3.4 Encapsulating Security Payload: Tunnel Mode

Authentication is only provided over ESP Header and Data, not over IP header fields. The entire payload is encrypted, meaning that the real source and destination addresses are hidden.

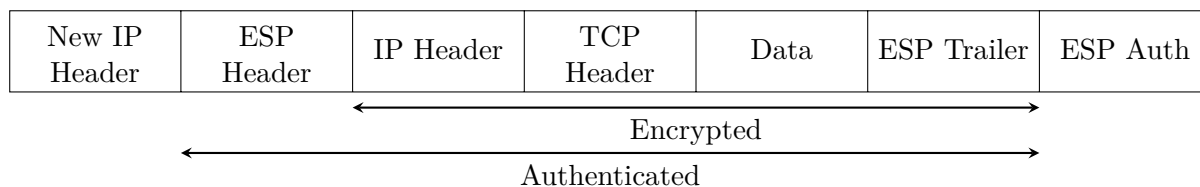


Figure 15.5: ESP Tunnel Mode

15.2 Virtual Private Networks

A *Virtual Private Network* (VPN) is a method of securely communicating data over an unsecured (or public) network. They are used in enterprise to extend connections to locations beyond the corporate trusted network. For example a teleworker uses a VPN to connect to the corporate network to access the file server, meaning the file server then doesn't have to be directly exposed to the internet.

VPNs must accomplish three things:

- Encapsulation of incoming and outgoing data
- Encryption of incoming and outgoing data (for secure VPNs)
- Authentication of remote computers and remote users

Similar to IPsec, a VPN can operate in either transport or tunnel mode.

15.2.1 Transport Mode VPN

Transport mode VPNs encrypt the data within the IP packet, but not the header of the packet; meaning that the origin and destination addresses are visible should the packet be inspected.

These VPNs allow for users to connect to a remote network directly from their workstation, with the network acting as though their workstation is directly connected to the remote network. This is the popular paradigm for remote access workers to use when connecting to their office network over the Internet, by connecting to a VPN server on the perimeter of the corporate network.

The VPN server acts as intermediate client and decrypts traffic coming from the client / encrypts traffic going to the client. The destination machine on the corporate network receives encrypted data and decrypts it.

15.2.2 Tunnel Mode VPN

Tunnel mode VPNs operate between two perimeter tunnel servers. These act as encryption points, encrypting all traffic that will traverse the unsecure network. On the receiving network, all traffic is decrypted by the perimeter tunnel server and released into the network in a decrypted state. This means that the end stations receive decrypted data.

The primary benefit of this model is that an intercepted packet reveals nothing about the true destination system.

15.3 Internet Key Exchange

The *Internet Key Exchange* (IKE) is a protocol used within IPsec to set up authenticated and encrypted communication channels between two devices which is used for VPN traffic. IKE is used to agree the parameters used in an IPsec communication - AH vs ESP, establish symmetric key, and cryptographic parameters used. IKE is the key management layer of IPsec. There are two versions: IKEv1 and IKEv2.

15.3.1 IKEv1

IKEv1 has two phases - the first creating the *IKE SA* (IKE Phase 1) tunnel between the two connected devices. The second creates an inner *IPsec SA* (IKE Phase 2) tunnel within the IKE Phase 1 tunnel. The data travels through this inner tunnel.

Phase 1 has two methods of operation: main mode and aggressive mode.



There are diagrams showing what is written out below available in the slides available on Moodle.

15.3.1.1 Phase 1: Main Mode

Main mode of phase 1 establishes a secure, authenticated and encrypted channel between peers using a six-message exchange:

1. A → B: Send the local IKE proposal
2. B → A: Confirm the IKE proposal
3. A → B: Send key-related information
4. B → A: Send key-related information
5. A → B: Send identity information
6. B → A: Send identity information

The first two messages are used to negotiate the parameters for establishing the IKE SA. A sends B the proposal of how it would like to communicate, offering a number of options, to which B responds with the chosen SA.

The third and fourth messages are used to exchange key related information and generate the key.

The final pair of messages, fifth and sixth, are used to exchange identity information to authenticate the peer.

See below '*Security Associations*' for more information on the HAGLE mnemonic and the process it outlines.

15.3.1.2 Phase 1: Aggressive Mode

Aggressive mode of phase 1 accelerates the same activities that main mode uses. This is used to establish tunnels faster, as it cuts down the number of required packets from 6 to 3. Commonly seen in the scenario where one peer has a dynamic IP address or where multiple clients connect to a single VPN gateway.

1. A → B: Send the local IKE proposal, key related information and identity information
2. B → A: Search for the matching IKE proposal. Sends the matching IKE proposal, key-related information, identity information and local authentication information
3. A → B: Responds with the local authentication information to implement authentication

Aggressive mode is less secure than main mode as it transmits identification information in plain text without encryption.

15.3.1.3 Phase 2

IKEv1 Phase 2, known as quick mode, is used to negotiate the security parameters of the inner IPsec tunnel which is used to protect the user data passing through the tunnels. It is a three-packet

configuration:

1. $A \rightarrow B$: Sends the IPsec parameters and identity authentication information
2. $B \rightarrow A$: Sends the matching IPsec parameters and identity information and generates keys
3. $A \rightarrow B$: Sends the confirmation information and complete phase 2 negotiation

15.3.2 Security Associations

During phase 1 of establishing a tunnel between two peers, there are 5 steps which must be undertaken. The steps can be remembered with the HAGLE mnemonic.

H Hash Algorithm - HMAC or Hashed Message Authentication Mode

A Authentication Method - how peers prove their identity (more below)

G Diffie-Hellman Group - determines the strength of the keys used in the key exchange process

L Lifetime - Defines how long the Phase 1 SA remains active before it must be re-negotiated

E Encryption Algorithm - Specifies the algorithm used to encrypt the data

Both sides must agree on the parameters to use and they must match, else the IPsec tunnel will fail to establish. This only applies to the Phase 1 connection - creating the management tunnel, not the phase 2 data tunnel.

15.3.2.1 Diffie-Hellman Key Exchange

The Diffie-Hellman (DH) key exchange uses a series of mathematical operations to identify a shared secret key. It uses the modular exponentiation (power mod), represented as MODP.

1. A and B agree modulus p and base g
2. A chooses secret integer a , then sends B: $X = g^a \mod p$
3. B chooses secret integer b , then sends A: $Y = g^b \mod p$
4. A computes $Z = Y^a \mod p$
5. B computes $Z = X^b \mod p$
6. A and B now share the secret Z

This works because $X^b \mod p = g^{ab} \mod p = g^{ba} \mod p = Y^a \mod p$

Example: DH Key Exchange

A and B agree modulus $p = 23$ and base $g = 5$

A chooses secret integer $a = 6$ then sends B:

$$X = g^a \mod p = 5^6 \mod 23 = 8$$

B chooses secret integer $b = 15$, then sends A:

$$Y = g^b \mod p = 5^{15} \mod 23 = 19$$

A computes:

$$Z = Y^a \mod p = 19^6 \mod 23 = 2$$

B computes:

$$Z = X^b \mod p = 8^{15} \mod 23 = 2$$

A and b now share a secret (the number 2).

15.3.2.2 Authentication

There are three different authentication options used.

The *Pre-Shared Key* option requires the two peers to have already established a shared key which they both know. They then undergo a six-message exchange to generate the session key and authenticate each other:

1. Initiator → Responder: HDR, SA
2. Responder → Initiator: HDR, SA
3. Initiator → Responder: HDR, KE, Ni
4. Responder → Initiator: HDR, KE, Nr
5. Initiator → Responder: HDR*, IDii, HASH-I
6. Responder → Initiator: HDR*, IDir, HASH-R

The first two messages are used for Initiator to tell the responder what possible Security Association (SA) options it can use, with the responder responding to confirm which of the options are suitable for it. The third and fourth messages contain the DH public keys (KE) and nonces (Ni and Nr) which are random numbers. This allows each side to then generate the shared secret without ever sending that over the connection. The final two messages are encrypted, hence the *, and contain the IDs (IDii and IDir) and a hash to prove they both know the pre-shared key (HASH-I and HASH-R).

An alternative to using pre-shared keys is to use *Signatures* for authentication. In a similar way to above, this operates a six-message exchange to authenticate each peer:

1. Initiator → Responder: HDR, SA
2. Responder → Initiator: HDR, SA
3. Initiator → Responder: HDR, KE, Ni
4. Responder → Initiator: HDR, KE, Nr
5. Initiator → Responder: HDR*, IDii, [CERT], SIG-I
6. Responder → Initiator: HDR*, IDir, [CERT], SIG-R

The first four messages are the same, it's the final two where there are differences. In message 5 and 6, a signature (SIG-I and SIG-R) are exchanged. This is the alternative to proving that they know the pre-shared key (i.e. by hashing it) as was completed above. The signature should match a hash which has been digitally signed using an agreed algorithm and the public key of the non-sending peer; so then the peer which receives the signature can decrypt using it's private key to then verify the sender. The optional certificate gets transmitted when the receiving peer doesn't already have the sending peer's public key. A peer is authenticated by extracting the public key from the certificate (or it's memory), validating that the certificate is signed by a trusted CA, then using that public key to decrypt the signature; if it matches then responder knows that the initiator is who the initiator says they are.

The third and final authentication method is using *Public Key Encryption*. This also uses a six-message exchange, however it is advances the identity proving to the third and fourth steps:

1. Initiator → Responder: HDR, SA

2. Responder → Initiator: HDR, SA
3. Initiator → Responder: HDR, KE, [HASH(1)], <IDii-b>PubKey-r, <Ni-b>PubKey-r
4. Responder → Initiator: HDR, KE, HDR, KE, <IDir-b>PubKey-r, <Nr-b>PubKey-i
5. Initiator → Responder: HDR*, HASH-I
6. Responder → Initiator: HDR*, HASH-R

The first two messages remain the same, with the initiator and responder negotiating parameters for the communication. The third message is different, however; the initiator takes the responders public key and uses it to encrypt it's own identity (IDii-b) and it's nonce (Ni-b). The responder echos this in message four, using the initiator's public key to encrypt it's own identity and nonce. This works here due to the fact that the responder (message 3) and initiator (message 4) are the only people holding the private key needed to decrypt the body of the message, and therefore this inherently proves their identity. Message 5 & 6 are used to transmit the hashes (HASH-I and HASH-R) of the keys to confirm both sides have derived the same key.

15.3.3 IKEv2

IKEv2 is a revised version of IKEv1. It revises 12 things:

1. Define entire protocol in 1 document, replacing 2 RFCs; incorporate changes to support NAT Traversal, Extensible Authentication and remote address acquisition
2. Simplify IKE by replacing the eight different initial exchanges with single four-message exchange
3. Remove the domain of interpretation, situation and labelled domain identifier fields, and the commit and authentication bits
4. Decrease IKE's latency in the common case by making the initial exchange 4 messages, and allowing piggybacking setup of Child SA on that exchange
5. Replace the cryptographic syntax for protecting IKE messages with one based closely on ESP to simplify implementation and security analysis
6. Reduce the number of possible error states by making the protocol reliable (i.e. all messages are acknowledges) and sequenced; has allowed for shortening creation of a child SA from 3 messages to 2
7. Increase robustness by allowing the responder to do not significant processing until it receives a message providing that the initiator can received messages at its claimed IP address
8. Fix cryptographic weaknesses such as the problem with symmetries in hashes used for authentication
9. To specify Traffic Selectors in their own payload type rather than overloading ID payloads; making more flexible the required Traffic Selectors
10. Specify required behaviors under certain error conditions or when data that is not understood is received in order to make it easier to make future revisions in a way that does not break backwards compatibility
11. Simplify and clarify how shared state is maintained in the presence of network failures and DoS attacks
12. Maintain existing syntax and magic numbers to the extent possible to make it likely that implementations of IKEv1 can be enhances to support IKEv2 with minimum effort.

The IKEv2 tunnel is a single layered tunnel and is established through a single four-message exchange:

1. $A \rightarrow B$: IKE SA initial exchange
2. $B \rightarrow A$: IKE SA initial exchange
3. $A \rightarrow B$: Authentication exchange
4. $B \rightarrow A$: Authentication exchange

The first two messages are used for IKE SA establishment and key generation. The final two messages are used to identify authentication and the establishment of the first pair of IPsec SAs.

Page 16

Lecture - Transport Layer Security

📅 2026-05-05

🕒 09:00

👤 Fahad

16.1 SSL/TLS

The *Secure Socket Layer* (SSL) aims to provide confidentiality and integrity between two communicating applications. *Transport Layer Security* (TLS) is a variant of SSLv3. SSL was originally designed for web environments by Netscape.

In this module, we'll focus on TLS 1.2 and 1.3.

These protocols provide a secure channel between the server and client. They provide server (and optionally client) authentication, confidentiality and message integrity. They run on top of the existing TCP protocol, making them transparent for application layer protocols. SSL/TLS connections act like secured TCP connections. Most protocols which run over TCP can be run over SSL/TLS instead, for example HTTP can be run on HTTPS, or SMTP can be run on SMTPS.

16.2 TLS

TLS provides a practical implementation of a hybrid encryption scheme with authentication and key exchange based on public keys. As the ciphers are negotiated in the handshake, it does not depend on the availability of any particular algorithms; nor does it depend on any secure services at the time of session establishment. The only requirement is that the public-key certificates are issued by an authority recognised by both parties.

16.2.1 Maintaining Security

As we have already seen, to implement and maintain security - we look for the modified CIA triad, this is no different within TLS.

Authentication is provided through ensuring that the server side of the channel is always authenticated; with the client side optionally authenticated. Authentication can happen via asymmetric cryptography using RSA, Elliptic Curve Digital Signature Algorithm (ECDSA), or Edwards-Curve Digital Signature Algorithm (EdDSA); or alternatively a symmetric pre-shared key (PSK).

Confidentiality is ensured through obfuscating the data while in transit so it is only visible to the endpoints. This can only happen after channel establishment, however. The data is obfuscated through tactics such as padding the TLS payload to obscure the length and improve protection against traffic analysis. The length is displayed to any threat actors, so padding is a good countermeasure especially when combined with authentication meaning the threat actor cannot see the length of the messages.

Through authentication and confidentiality comes integrity. The data sent over the channel after establishment cannot be modified by attackers without detection.

16.2.2 Components of TLS: A Summary

TLS consists of two key components, a handshake protocol and record protocol.

A *handshake protocol* covers the following:

- establishes and maintains a TLS session (a secure channel) between a client and a server
- authenticates the communicating parties (the client & server)
- negotiates cryptographic modes and parameters
- establishes shared keying material

The handshake protocol is designed to resist tampering. This means that an attacker should not be able to force the peers to negotiate different parameters than they would if the connection were not under attack.

A *record protocol* covers the following:

- data transfer, compute MAC for integrity and encrypt MAC and data for confidentiality
- use parameters established by handshake protocol to protect traffic between the communicating peers

The record protocol divides traffic into a series of records, each of which are independently protected using the traffic keys.

16.2.3 Record Protocol

The TLS record protocol is a session-level protocol. It transport the application data transparently between a pair of processes, guaranteeing the confidentiality, integrity and authenticity of the data. The communicating peers can deploy decryption and authentication of messages in each direction, with each peer being able to store session identifiers for subsequent reuse, avoiding the overhead of establishing a new session.

The record protocol adds a secure comms layer below existing application-level protocols which is typically used to secure HTTP interactions for internet commerce and other security-sensitive applications. It is implemented by virtually all web browsers and web servers.

The protocol prefix **https:** in URLs initiates the establishment of a TLS secure channel between a browser and a web server.

When transmitting, the message size for transmission is first fragmented into blocks of a manageable size, then the blocks are optionally compressed. Compression is not strictly a feature of secure communication, but it is provided here because a compression algorithm can usefully share some of the work of processing the bulk data with the encryption and digital signature algorithms.

A pipeline of data transformations is set up within the TLS record layer that performs required transformations more efficiently than could be done independently. Then *encryption and message authentication* (MAC) transformations deploy agreed cipher suite algorithms (see below for more on this). Finally, the signed and encrypted block is transmitted to the peer through associated TCP connection, where transformations are reversed to produce original block data.

16.2.4 Master Secret

Regardless of which key exchange method is used in TLS, the same function calculates the ‘pre_master_secret’ into the 48 byte ‘master_secret’.

$\text{PRF}(\text{pre_master_secret}, \text{‘master secret’}, \text{ClientHello.random} + \text{ServerHello.random})$ [0..47];

When using RSA (TLS_RSA_), a ‘pre_master_secret’ is generated by the client which is then encrypted by the server’s public key and send to the server. The server uses its private key to decrypt the ‘pre_master_key’.

When using Diffie-Hellman (TLS_DHE_, TLS_ECDHE_, etc), the DH negotiated shared secret is used as the ‘pre_master_secret’. DH parameters are specified by the server and may either be

ephemeral or fixed and certified.

16.2.5 TLS Cipher Suites

TLS supports a variety of options for the cryptographic functions to be used. Collectively known as a ‘cipher suite,’ they include a single choice for each of the following:

- Key Exchange Algorithm (and means of authentication) which establishes bulk encryption key
- Bulk Encryption Algorithm - encrypts application data following handshake
- MAC algorithm - used to authenticate and integrity check the message

Example: TLS Cipher Suite

For example, the suite:

`‘TLS_RSA_WITH_3DES_EDE_CBC_SHA’`

specifies:

- Key Exchange - RSA
- Bulk Encryption - Triple DES - Cipher Block Chaining Mode
- MAC - Secure Hash Algorithm 1

Alternatively, the suite:

`‘TLS_DHE_RSA_WITH_AES_256_CBC_SHA256’`

specifies:

- Key Exchange - Ephemeral Diffie-Hellman with RSA signature
- Bulk Encryption - Advanced Encryption Standard - Cipher Block Chaining mode
- MAC - SHA256

A variety of popular cipher suites are preloaded with standard identifiers in the client and the server. During the handshake, the client lists a preference of accepted cipher suites and the server responds by selecting one of them (or giving an error indication if it has none that match). The client and server also agree on a (optional) compression method and a random start value for CBC block encryption functions.

Upon completion of the handshake, subsequent communication is encrypted and signed according to the agreed cipher suite and key.

16.2.6 TLS Handshake

The TLS handshake protocol is performed over an existing connection and begins in the clear (i.e. not encrypted) with the aim of establishing a TLS session by exchanging options / parameters needed to perform encryption and authentication. The handshake sequence varies depending on what / how authentication and key exchange are achieved.

The handshake protocol can also be invoked at a later time to change the specification of a secure channel. For example, communication may begin with message authentication using MACs only and then later encryption needs to be added. This is achieved by performing the handshake protocol again to negotiate a new cipher specification using the existing channel.

The message flow for a full handshake can be seen below:

1. Client → Server: ClientHello

2. Server → Client: ServerHello
3. Server → Client: Certificate
4. Server → Client: CertificateRequest*
5. Server → Client: ServerHelloDone
6. Client → Server: Certificate*
7. Client → Server: CertificateVerify*
8. Client → Server: Change Cipher Spec
9. Client → Server: Finished
10. Server → Client: Change Cipher Spec
11. Server → Client: Finished

The first two messages establish the protocol version, session ID, cipher suite, compression method and exchange random values used in the cryptography. The third, fourth and fifth messages optionally send the server certificate and request the client certificate then the server confirms it's done sending. The sixth and seventh messages are used for the client to respond with it's certificate and verify the server's provided certificate. The final four messages then change the cipher suite to the suite established during the handshake and confirm the handshake is finished handshake.

The steps of the TLS handshake are shown below, including parameters of each step.

Client → Server: ClientHello

- Version: highest TLS protocol version supported by the client
- ClientRandom: random number
- Session ID: when resuming a session [optional]
- Cipher suites: prioritised list of all supported cipher suites

Server → Client: ServerHello

- Version: highest TLS version supported by the server (and client)
- ServerRandom: random number
- Session ID: implementation specific, random number
- Cipher Suite: Chosen by server from list sent by client. The single strongest cipher suite supported by both server and client (else a handshake failure alert is created)

Server → Client: Certificate

- Server sends certificate to client, client obtains server's public key and verifies certificate

Server → Client: ServerKeyExchange

- for DHE: modulus, generator, public value
- for ECDHE: prime, constants, base, order and cofactor [optional]
- for RSA: none

Server → Client: Certificate Request [optional]

Server → Client: ServerHelloDone

- Marks the end of server messages

Client → Server: ClientKeyExchange

- for DHE: modulus, generator, public value
- for ECDHE: prime, constants, base, order and cofactor [optional]
- for RSA: random value encrypted with server public key

Client → Server: CertificateVerify [optional]

Client → Server: ChangeCipherSpec

- Notify that the client switched to the new CipherSuite

Client → Server: Finished

- Encrypted final message containing hash over the previous handshake message

For DHE and ECDHE both the client and server compute the joint session key based on the parameters provided. For RSA - this value is provided by the client to the server.

Server → Client: ChangeCipherSpec

- notify that server switched to a new CipherSuite

Server → Client: Finished

- Encrypted final message containing the hash over the previous handshake message

16.2.7 Resuming an Interrupted TLS Session

Interrupted sessions can resume if needed. The client and server store both the session ID and MasterSecret. Then the following steps can be undertaken:

- Client sends session ID in ClientHello
- Reduced protocol followed: Hello, ChangeCipherSpec and Finished messages used only (they already have encryption established previously)
- New keying data is exchanged
- New session keys are derived

16.2.8 Vulnerabilities in TLS

There have been a number of vulnerabilities in TLS over the years including Heartbleed, POODLE, FREAK, SKIP-TLS, Logjam.

Heartbleed plays on the fact that the TLS heartbeat extension keeps the client and server connected even if the connection isn't currently being actively used. Occasionally, one peer will send an encrypted piece of data (called a heartbeat) to the other peer. The receiver will reply with the same encrypted piece of data, proving that the connection is still in place. The heartbeat request includes information about its own length.

It's in this 'information about its own length' we find the flaw. OpenSSL's implementation failed to check that the request was the claimed length. If a request claimed to be 40KB but was only 20KB, the receiver would set aside 40KB of memory buffer then store the 20KB that it actually received. It would then return the 20KB of data it received, plus whatever happened to be in the next 20KB of memory. This had the potential to spill secrets from the contents of memory.



There are links to information about the other vulnerabilities in the slides on Moodle.

16.2.9 TLS 1.3 Differences to TLS 1.2

- Support for legacy symmetric encryption algorithms has been pruned
- Cipher suite concept changed to separate the authentication and key exchange mechanisms from the record protection algorithm, and a hash to be used with both the key derivation function and handshake message authentication code (MAC)
- A zero round-trip time (0-RTT) mode was added, saving a round trip at connection setup for some application data, at the cost of certain security properties
- Static RSA and DH cipher suites have been removed. All public-key based key exchange mechanisms now provide forward secrecy
- All handshake messages after the ServerHello are now encrypted. The newly introduced EncryptedExtensions message allows various extensions previously sent in the clear in the ServerHello to be protected
- The key derivation functions have been redesigned - allowing easier analysis by cryptographers due to their new key separation properties. The HMAC-based Extract-and-Expand Key Derivation Function (HKDF) is used as an underlying primitive
- The handshake state machine has been significantly restructured to be more consistent and to remove superfluous messages such as ChangeCipherSpec
- Elliptic curve algorithms are now in the base spec
- Other miscellaneous cryptographic improvements including changing the RSA padding to use the RSA Probabilistic Signature Scheme (RSASSA-PSS) and the removal of compression, the Digital Signature Algorithm and custom Ephemeral DH groups
- TLS 1.2 version negotiation mechanism has been deprecated in favour of a version list in an extension
- Session resumption with and without server-side state as well as the PSK based cipher suites of earlier TLS versions have been replaced by a single new PSK exchange
- References have been updated to point to the updated versions of RFC



The TLS message flow for a full handshake can be found in the slides on Moodle.

Page 17

Lecture - Application Layer Security

📅 2026-05-12

🕒 09:00

👤 Fahad

There is information on Moodle relating to the final exam.

17.1 Needham Schroeder Symmetric Key Protocol

The *Needham Schroeder Symmetric Key* (NSSK) protocol is now obsolete but is a theoretical underpinning concept for Kerberos, talked about later in this lecture.

NSSK uses a *Key Distribution Centre* (KDC) to distribute shared session keys using only symmetric key cryptography. This works through each party, A and B independently holding a shared symmetric key with the KDC, S in advance. A holds K_{as} and B holds K_{bs} .

Then when A and B want to communicate, the KDC helps establish a fresh shared key between them, K_{ab} . This is generated using single-use random number (Nonces), N_a and N_b , which are used to help authenticate the parties. The message flow can be seen below:

1. $A \rightarrow S: A, B, N_A$
 A requests S to supply a key for communication with B .
2. $S \rightarrow A: \{N_a, B, K_{ab}, \{K_{ab}, A\}_{K_{bs}}\}_{K_{as}}$
 S returns a message encrypted in A 's secret key (K_a) containing the newly generated K_{ab} and a ticket encrypted in B 's secret key. The nonce N_a demonstrates that the message was sent in response to the preceding one. A believes that S sent the message because only S knows A 's secret key.
3. $A \rightarrow B: \{K_{ab}, A\}_{K_{bs}}$
 A sends the ticket to B .
4. $B \rightarrow A: \{N_b\}_{K_{ab}}$
 B decrypts the ticket and uses the new key, K_{ab} to encrypt another nonce, N_b .
5. $A \rightarrow B: \{N_b - 1\}_{K_{ab}}$
 A demonstrates to B that it was the sender of the previous message by returning an agreed transformation of N_b .

There are, however, weaknesses with NSSK. Message 3 is weak, in that B has no reason to believe the message is fresh; there's no nonce to prove this. If an intruder was to obtain K_{ab} and $\{K_{ab}, A\}_{K_{bs}}$ then the intruder can initiate in a subsequent exchange with B impersonating A . For this attack to occur, an old value of K_{ab} has to have been compromised.

This weakness can be remediated by adding a nonce (timestamp) to message 3. Message 3 would then become $\{K_{ab}, A, t\}_{K_{bs}}$. B decrypts this message and checks that t is recent. This is the solution that has been adopted in Kerberos.

17.2 Kerberos

Yup, this has been covered in IT & Internetworking Security, and will be repeated here. Both slightly differently.

Kerberos was developed at MIT in the 1980s to provide authentication and security services for the MIT campus computing network. It has gone through a number of revisions and is widely adopted. In this lecture, we'll consider Kerberos version 5 as specified in RFC 4120.

Kerberos involves three security objects:

Definitions

Ticket A token issued to a client by the *Kerberos Key Distribution Centre* (KDC) to present to a particular server, verifying that the sender has recently been authenticated by Kerberos. The ticket includes an expiry time and a newly granted session key for use by the client and the server.

Authenticator A token constructed by a client and sent to a server to prove the identity of the user, and also is the currency of any communication with a server. An authenticator can be used only once. It contains the client's name and a timestamp, and is encrypted using the appropriate session key.

Session Key A secret key randomly generated by Kerberos and issued to a client for use when communicating with a particular server. Encryption is not mandatory for all communication with servers; the session key is used for encrypting communication with those servers that demand it, and for encrypting all authenticators.

A client must possess a ticket and session key for each server that they use. The tickets are granted to clients with a lifetime of several hours (at the cyber security team's discretion) so that they can be used for interaction with a particular server until they expire, at which point re-authentication is required.

Each *Key Distribution Centre* (KDC) contains within it an *Authentication Service* (AS) and a *Ticket Granting Service* (TGS). On login, users are authenticated by the AS. The client is supplied with a TGS ticket and a session key for subsequent communication with the TGS. The client can use the TGS ticket to obtain tickets and session keys for specific services, called *server keys*, from the TGS.

17.2.1 Kerberos' Relationship to NSSK

As alluded to above, Kerberos is related to NSSK. Kerberos follows NSSK quite closely, however Kerberos adds more Nonces which serves two purposes:

- To guard against replays of old messages intercepted in the network or the reuse of old tickets found in the memory of machines from which the authorised user has logged out; and
- To apply a lifetime to tickets, enabling the system to revoke user's rights if they cease to be authorised users of the system.

17.2.2 Authentication Process



There's a diagram of the following Kerberos process in the slides on Moodle.

The below steps show how Kerberos works:

Part A: Obtain Kerberos Session key & TGS ticket, happens once per login session

1. $C \rightarrow A: C, T, n$

Client C requests the Kerberos authentication server A to supply a ticket for communicating with the ticket-granting service T .

2. $A \rightarrow C: \{K_{ct}, n\}_{K_c}, \{ticket(C, T)\}_{K_t}$

AS returns a message containing a ticket encrypted in its secret key and a session key for C to

use with T . The inclusion of nonce n encrypted in K_c shows that the message comes from the recipient of message 1, who must know K_c . The ticket contains C, T, T_1, t_2, K_{ct} where t_1 and t_2 are the start and end time for the ticket to be valid.

Part B: Obtain ticket for a server, once per client-server session

3. $C \rightarrow T: \{auth(C)\}_{K_{ct}}, \{ticket(C, T)\}_{K_t}, S, n$
 C requests the ticket granting server, T , to supply a ticket for communication with another server, S .
4. $T \rightarrow C: \{K_{cs}, n\}_{K_{ct}}, \{ticket(C, S)\}_{K_s}$
 T generates the ticket. if it is valid, T generates a new random session key, K_{cs} , and returns it with a ticket for S (encrypted in the server's secret key, K_s).

Part C: Issue a server request with a ticket

5. $C \rightarrow T: \{auth(C)\}_{K_{cs}}, \{ticket(C, S)\}_{K_s}, request, n$
 C sends the ticket to S with a newly generated authenticator for C and a request. The request would be encrypted in K_{cs} if secrecy of the data is required.

Part D: Client authenticates server [optional]

6. $S \rightarrow C: \{n\}_{K_{cs}}$
 S sends the nonce to C , encrypted in K_{cs} .

17.2.3 Using Kerberos to Login

When a user logs into a workstation, the login program sends the user's name to the Kerberos *Authentication Service* (AS). If the user is known to the AS, it replies with a session key, a nonce encrypted in a shared key derived from the user's hashed password, and a ticket for the *Ticket Granting Service* (TGS).

The login program then attempts to decrypt the session key and the nonce using a key derived from the password that the user typed in response to the password prompt. If the password is correct, the login program obtains the session key and the nonce. It checks the nonce and stores the session key with the ticket for subsequent use when communicating with the TGS.

The login program can erase the user's password from its memory, since the ticket now serves to authenticate the user. A login session is started for the user on the workstation.

Note that the user's password is never exposed to eavesdropping on the network, it is retained wholly within the workstation.

17.2.4 Accessing Servers with Kerberos

Whenever a program running on a workstation needs to access a new service, it requests a ticket for the service from the TGS.

When a UNIX user wishes to log into a remote computer, the *rlogin* command prompt on the user's workstation obtains a TGS ticket for access to the *rlogind* network service. The *rlogin* command program sends the TGS ticket, together with a new authenticator, in a request to the *rlogind* process on the computer where the user wishes to login. The *rlogind* program decrypts the TGS ticket with the *rlogin* service's secret key and checks its validity. The *rlogind* program then uses the session key included in the ticket to decrypt the authenticator and checks it is fresh.

There is no need to check the user's name and password following these checks, as the user's identity is known to the *rlogind* program and a login session is established for that user on the remote machine.

17.3 Access Control Policies

Access privileges are specified and subjects' access to objects are determined through a security policy.

Definitions

DAC Discretionary Access Control Policy - controls access based upon identity

MAC Mandatory Access Control Policy - controls access based upon security labels

RBAC Role-Based Access Control Policy - controls access based on roles

ABAC Attribute Based Access Control Policy - controls access based on attributes

17.4 SAML

The OASIS *Security Association Markup Language* (SAML) standard defines an XML-based framework for describing and exchanging security information between online business partners. This security information is expressed in the form of portable SAML assertions that applications working across the boundaries of security domains can trust. The OASIS SAML standard defines precise syntax and rules for requesting, creating, communicating and using these SAML assertions. SAML is behind many 'single sign on' applications. It can be seen with the `sso.port.ac.uk` authentication method which UoP uses.

17.4.1 Single Sign On

Single Sign On (SSO) is the idea of logging in once and this identity being usable across multiple different applications, websites, or service.

The single identity which exists across multiple applications is known as a *federated identity* and it will be based on a business arrangement. The *identity provider* (IdP) asserts to the *service provider* (SP) that:

- the user is known (by referring to the user by their federated identity)
- has authenticated to it
- and has certain identity attributes

SAML 2.0 supports numerous variations on the two main flows (as below) that deal with requirements for:

- using various types and strengths of user authentication methods
- alternative formats for expressing federated identities
- use of different bindings for transporting the protocol messages
- inclusion of identity attributes

The two main flows are shown below.

17.4.1.1 SP Initiated SSO

SP initiated SSO begins with the user navigating to an SP resource they need to access. As the user is not currently logged in, the SP notices this and sends the user to an IdP to authenticate. The IdP builds an assertion representing the user's authentication at the IdP and sends the user back to the SP with the assertion. The SP processes the assertion and determines whether to grant the user access to the resource.

17.4.1.2 IdP Initiated SSO

In this flow, the user has already authenticated to an IdP and they navigate to a partner SP. The IdP builds an assertion representing the user's authentication state at the IdP and sends the user's browser over to the SP's assertion consumer service. The SP processes the assertion and creates a local security context for the user at the SP. This requires the IdP to be configured with links to partner SP sites.

17.4.2 Federated Identity

SAML can dynamically establish a federated identity during web SSO. Local identities must be linked to the federated identity used to represent the user when interacting with a partner.

The process of associating a federated identifier with the local identity at a partner is called account linking.

Example: Federated Identity Management

In this example there is one IdP: `airline.example.com`, and two SPs: `cars.example.com` and `hotels.example.com`. The user is registered on all three provider sites but the local accounts all have different account identifiers:

- `airlines.example.com` - John is registered as *johndoe*
- `cars.example.com` - John is registered as *jdoe*
- `hotels.example.com` - John is registered as *johnd*

The sites have agreed to use persistent privacy-preserving pseudonyms for the user's federated identities. John hasn't previously federated identities between these sites.

John starts off his very expensive endeavour by booking flights at `airline.example.com`. He uses his *johndoe* account.

John then visits `cars.example.com` to reserve a car. The site sees the user isn't logged in locally, but has previously visited an IdP partner site (`airline.example.com`). So `cars.example.com` asks John for consent to federate identities. John consents and his browser redirects back to `airline.example.com` where the site creates a new pseudonym (*azqu3H7*) for John's use when he visits `cars.example.com`. The pseudonym is linked to his *johndoe* account.

John is then redirected to `cars.example.com` with a SAML assertion indicating user *azqu3H7* is logged in at the IdP. As this is the first time that `cars.example.com` has seen this identifier, it doesn't know which local account it refers to. John must login at `cars.example.com` using his *jdoe* account. Then `cars.example.com` attaches the identity *azqu3H7* to the local *jdoe* account for future use with the IdP.

After reserving a car, John heads to `hotels.example.com` in order to book a hotel room. The federation process is repeated with the IdP `airline.example.com`, creating a new pseudonym (*f78q9C0*) for IdP user *johndoe* that will be used when visiting `hotels.example.com`.

John is redirected back to the `hotels.example.com` SP with a new SAML assertion. The SP requires John to log into his local *johnd* user account and adds the pseudonym as the federated name identifier for future use with the IdP `airline.example.com`. The user accounts at the IdP and this SP are now linked using the federated name identifier *f78q9C0*.

`hotels.example.com` does not know about the federation with, or John's account on, `cars.example.com`. Equivalently, `cars.example.com` does not know about John's account on or federation with `hotels.example.com`.

In the future, when John needs to make another booking, he will only need to log in once at `airline.example.com`.

FREELY GIVEN, HIGHLY QUESTIONABLE

These notes are a byproduct of my learning process - mistakes, bad jokes and snark included. If you take an error of mine as gospel and ruin your perfect 1st, that is a *you* problem boo. I am a student, not a prophet, use a textbook if you want certainty.

Yes, they have been typeset in $\text{\LaTeX}$; I'd rather typeset in stone with a blunt chisel than use Word Online for another minute. If the formatting looks obsessive, it's because it was either this or a breakdown. You can find the source on my [GitHub](#).

They're licensed under CC-BY-4.0. So do what you will with them; just attribute them to me. Plagiarism is a bad look, and I've got just enough snark left for you bestie.

Thomas Boxall is a Computer Science Student studying at the University of Portsmouth, UK. In all honesty, he should probably be revising or doing some work towards his dissertation, not fondling the template of his notes again. You can find Thomas online at thomasboxall.net